

CHIRP — Complete Project Extraction

Cuckoo-inspired Hierarchical Intrusion-Resilient Protocol for VANETs

BTP project extraction, reconstructed from the repository at commit `07c9aec` (branch `master`, working tree clean apart from one untracked mirror file). Every number in this document was read out of `results/*.json` or produced by re-running the tools in `src/`; where a document and a data file disagree, the disagreement is reported rather than resolved silently.

PART 1 — COMPLETE PROJECT IDENTITY

1.1 Title

CHIRP — Cuckoo-inspired Hierarchical Intrusion-Resilient Protocol for VANETs.

1.2 One sentence

We reimplement a published energy-efficient VANET clustering protocol (CSGD-NET), extend it with mobility, trust, cryptography and emergency-priority traffic, and in doing so discover that priority handling is itself an attack surface that only cryptographic authorisation can close.

1.3 The research problem, in full

Vehicular ad hoc networks (VANETs) let vehicles and roadside units exchange data. A large literature makes them **energy-efficient** by *clustering*: vehicles group together, elect a cluster head (CH), send their readings to that head, and the head **fuses** them into one packet which it forwards to a roadside unit (RSU). Fusion is what saves energy — one transmission leaves the cluster instead of ten.

Three things are missing from that literature, and they interact:

1. **No security.** The base protocol has no authentication, no trust, no adversary model, and no attack evaluation. Its cluster-head election reads only self-reported residual energy and distance, so a node that lies wins the head role and can then drop whatever it likes.
2. **No mobility.** The base protocol assumes a static topology — in a *vehicular* network. Its own §4.3 concedes this.
3. **No notion of message criticality.** All traffic is homogeneous periodic telemetry. There is no emergency class, no deadline, and no priority.

The third gap is not a missing feature. It is a **structural conflict**. Fusion requires the head to wait for every member's TDMA slot before it can transmit, so every reading in a cluster inherits the delay of the whole frame. An ambulance's message cannot wait for the frame, and cannot be averaged into a fused packet with ten telemetry readings. **The mechanism that provides the energy saving is hostile to the traffic that matters most.**

1.4 Why it matters

Emergency vehicle communication is a safety function. A "green corridor" request, an approaching-ambulance warning, or a collision alert has a deadline; telemetry does not. If a protocol optimised for energy silently imposes whole-frame latency on safety traffic, it is unfit for the application it claims to serve — and nobody notices, because nobody measures a metric that has a deadline in it.

Second, once you *do* give emergency traffic a fast path, that fast path is a finite reserved resource, and a resource that a cluster head cannot authenticate. A vehicle that falsely claims emergency status takes an ambulance's slot. Priority becomes a denial-of-service surface.

1.5 What the base paper does

CSGD-NET uses **Cuckoo Search** with **Lévy flights** and a **Gaussian random walk** to select the cluster-head set, scoring candidates on residual energy and distance to the RSU. See Part 2.

1.6 Limitations we identified

#	Limitation	Where
1	No security whatsoever	security named as a challenge in §2, never addressed
2	Static topology in a vehicular network	§3.3, §4.1; conceded in §4.3
3	Fitness reads only residual energy and distance-to-RSU — a liar wins CH	Eq. 9
4	CH-aggregated data never integrity-checked	§3.3.2
5	50×50 m field, 50 rounds, no PDR/delay/overhead metrics	Table 3, Figs. 11–15
6	Eq. 9 described as a minimisation but only works as a maximisation	§3.3.1 vs Table 4
7	Eq. 9 does not determine an implementation — four defensible readings give first-node-death from 9.6 to 22.2 rounds	our finding, docs/reproducibility.md
8	Table 4's LEACH-C and CSGD-NET columns are not producible by the paper's own equations	our finding, src/verify_table4.py
9	No cluster-formation protocol at all — no ADV, no JOIN, membership never specified	our reading of §3.3.2
10	No downlink; traffic is strictly uplink	our reading
11	TDMA within clusters, nothing between them (LEACH pairs TDMA with CDMA; CSGD-NET never mentions CDMA)	our reading

1.7 What our project adds

- **Module 1** — mobility, and a six-term cluster-head objective that includes the intra-cluster distance term Eq. 9 omits.
- **Module 2** — a three-layer Beta-reputation trust engine, and six adversary behaviours to test it against.
- **Module 3a** — Scyther formal verification of the authorisation layer.
- **Module 3b** — measured cryptographic cost, in bytes, microseconds and energy.
- **Module 5** — a second traffic class with a deadline, a reserved fast path that bypasses fusion, the false-priority attack against it, and the authorisation that closes it.
- **Cross-cutting** — a reproduction audit of the base paper, and a regression gate that keeps all of the above from silently corrupting the reproduction.

Module numbering is historical. **Module 4 (RSU-side IDS) does not exist** (verified — see Part 30). **Module 5 lives in docs/priority.md**, not docs/module5.md.

1.8 Research hypothesis

Adding mobility-awareness, trust and cryptography to an energy-optimised VANET clustering protocol will make it usable in a realistic adversarial vehicular setting, at an energy cost small enough to be worth paying.

That hypothesis was **partly confirmed and partly overturned**. Confirmed: the costs are small (1–2 % of network energy). Overturned: trust turns out to be insufficient in a way we did not anticipate, and the energy metric turns out to be actively misleading as a security metric.

1.9 Research question

What does it actually cost — in energy, in latency, and in security — to carry deadline-bound emergency traffic inside an energy-optimised clustering protocol, and what mechanism is strictly necessary to protect that traffic?

1.10 What the final system demonstrates

1. Aggregation and deadlines are in direct opposition, and the opposition is measurable.
2. The fast path that resolves it is a finite resource, and therefore a DoS surface.
3. Behavioural detection **bounds** abuse of that resource but cannot stop it.
4. An authenticated role attribute stops it completely, at zero cost when no attack is present.

5. The cost of the crypto that provides it is dominated by *computation*, which the base paper's energy model is structurally incapable of showing.

1.11 The same thing, in plain language

Cars form groups. One car in each group collects everyone's data, squashes it into a single message, and sends it to a roadside box. Squashing saves battery, which is the whole point of the protocol we started from.

The problem: to squash, the group leader has to wait for everyone to speak. That's fine for routine data. It is not fine for an ambulance.

So we gave emergency messages a reserved fast lane that skips the squashing. That works — emergency messages always arrive on time, and it costs about 1–2 % of the network's battery.

But a reserved lane is a limited resource, and the group leader has no way to check whether the car claiming to be an ambulance really is one. So a liar can take the ambulance's lane. We measured that: with 20 % of cars lying, real ambulances miss about 44 % of their deadlines and 11 % of the whole network's battery is spent carrying fake-urgent messages.

We tried to catch the liars by watching their behaviour. It half worked, and then failed in an interesting way: a patient liar who behaves *exactly like* a real ambulance is invisible to any behavioural test, and still does about half the damage.

The only thing that works is a certificate — a signed credential saying "this vehicle really is an ambulance". With that, the attack disappears completely. We proved the certificate protocol correct with a formal verification tool, and that tool found a real bug in our design that our simulator could not see.

PART 2 — THE BASE PAPER

2.1 Identification

- **Title:** *Optimizing vehicular networks communications through green clustering and data aggregation*
- **Authors:** Lamaa Sellami, Abir Mchergui, Bechir Alaya
- **Venue/Year:** *Cluster Computing* **29:16 (2026)**
- **DOI:** 10.1007/s10586-025-05776-1
- **Protocol:** **CSGD-NET** — Cuckoo Search with Gaussian Distribution

2.2 What CSGD-NET proposes

Cuckoo Search where a "nest" is a candidate cluster-head *set*. Each iteration:

1. A new candidate is drawn by **Lévy flight** (heavy-tailed random step).
2. If it does not improve the objective, a second candidate is drawn by a **Gaussian random walk** (local refinement).
3. A fraction `pa` of the worst nests is abandoned and re-randomised.

The Gaussian walk is the paper's stated contribution over plain Cuckoo Search: Lévy flights give global exploration, the Gaussian walk gives local exploitation.

2.3 Network model

- 100 nodes, randomly distributed, **50 × 50 m** ("Simulation Zone", Table 3).
- Base station / RSU at the **centre of the detection area**.
- **All nodes stationary** throughout the simulation.
- Three node types: cluster heads, ordinary members, RSUs.
- **Assumption of full RSU coverage** — every OBU can reach the RSU directly, no connectivity gaps. This assumption is load-bearing: it is why no routing is needed.
- Computation and processing are assumed to consume **no energy** (§3.3); only transmission and reception cost.

2.4 Energy model — Eqs. (9)–(11)

Standard first-order (Heinzelman) radio:

$$\begin{aligned} E_{TX}(k, d) &= k \cdot E_{elec} + k \cdot \epsilon_{fs} \cdot d^2 && \text{if } d \leq d_0 \\ &= k \cdot E_{elec} + k \cdot \epsilon_{mp} \cdot d^4 && \text{if } d > d_0 \\ E_{RX}(k) &= k \cdot E_{elec} \\ d_0 &= \sqrt{\epsilon_{fs} / \epsilon_{mp}} \end{aligned}$$

Table 3 parameters (units omitted in the paper; they are the conventional Heinzelman values and we adopt them):

symbol	value	our config field
E_elec	50 nJ/bit	e_elec = 5e-08
ε_fs	10 pJ/bit/m²	eps_fs = 1e-11
ε_mp	0.0013 pJ/bit/m⁴	eps_mp = 1.3e-15
packet	6400 bit	packet_bits = 6400
nodes	100	n_nodes = 100
CH %	10	ch_percent = 0.1 → n_ch = 10
initial energy	"1.01 d" (unreadable); Table 4 starts at 0.5	initial_energy = 0.5

d0 = 87.71 m. In a 50×50 m field (diagonal 70.7 m) the free-space d² branch applies everywhere; the d⁴ branch never fires. This matters enormously — see Part 16.

Two parameters the paper never states, which we had to supply:

- **E_DA (aggregation cost).** Not given. We use the standard LEACH value 5 nJ/bit/signal (`e_da = 5e-09`).
- **TDMA frames per round.** Not given. Fitted to Table 4's LEACH column at **22** (`packets_per_round = 22`); see Part 16.

2.5 Clustering, CH selection, routing

CH selection is Eq. 9:

$$f = E / (E_0 \cdot \text{dis}(\text{CH}_i, \text{RSU}))$$

where the paper's symbol list defines *E* as "the overall residual energy within the vehicular network" and *dis* as the CH-to-RSU distance. The prose says the objective "targets the minimization"; the printed expression must be **maximised** (residual energy in the numerator, distance in the denominator). That min/max wording is presentational — maximising the printed form equals minimising its reciprocal, so every implementer lands on the same ranking.

Cluster formation is never specified. The paper does not describe how a vehicle discovers or joins a head — no advertisement, no join request. LEACH's ADV → JOIN-REQ → schedule handshake appears only in the paper's *related-work* description of LEACH, never for CSGD-NET. So our nearest-head association rule is **our assumption, not an inherited one**, and is disclosed as such.

Data path, from §3.3.2:

- Member → CH over **TDMA**; members transmit in designated slots and stay inactive between them. TDMA is stated to prevent collisions *within* a cluster.
- CH **consolidates/aggregates** and transmits **directly to the RSU** — single hop.
- An **isolated vehicle** with no cluster association sends **directly to the RSU**, bypassing CHs. (This is what we call an *orphan*, and it becomes central to Part 8.)
- The RSU relays onward to other RSUs or external networks — outside the energy model.

There is no multi-hop routing. Despite the title, the topology is strictly two hops: member → CH → RSU. No CH-to-CH forwarding, no route discovery. It is a clustering-and-aggregation protocol with a routing-protocol name. This matters when comparing against related work, all of which does multi-hop.

2.6 Evaluation and reported results

Metrics: residual energy per round, alive nodes, packets delivered. No PDR, no delay, no overhead, no confidence intervals (only Fig. 14 mentions a 95 % interval). Comparison protocols: LEACH, LEACH-C, HEED, PSO, GA.

Table 4 — mean residual energy per node (the paper's headline table):

Round	LEACH	LEACH-C	CSGD-NET	HEED	PSO	GA
1	0.5	0.5	0.5	0.5	0.5	0.5
10	0.35529	0.37000	0.32982	0.41	0.415	0.42
20	0.22091	0.28339	0.26095	0.32	0.33	0.34
30	0.12717	0.14785	0.15064	0.23	0.245	0.26
40	0.039543	0.043871	0.074354	0.14	0.16	0.18

2.7 What we reproduced, extended, and dispute

Reproduced faithfully: the radio model, the Cuckoo Search engine (Lévy flight via Mantegna's algorithm, Gaussian walk, abandonment), the TDMA steady-state phase, the orphan rule, and the five comparison protocols.

`src/protocols/csgd_net.py` is a direct implementation of §3.3 and Algorithm 2.

Extended: mobility (Module 1), trust (Module 2), crypto (Module 3), priority (Module 5), plus setup-phase energy accounting which the paper omits entirely.

Disputed, with evidence: Eq. 9's under-determination, and Table 4's LEACH-C/CSGD-NET columns. Both are Part 16.

2.8 How our implementation relates to the original

CHIRP subclasses CSGDNet and inherits the Cuckoo engine byte-for-byte — same Lévy flight, same Gaussian walk, same abandonment, same iteration count. Only the objective and the re-clustering trigger differ. Every protocol runs on identical mobility traces (same seed → same road, same speeds).

Any measured difference is therefore attributable to the objective, not to a different optimiser. That is the point of subclassing rather than rewriting, and it is a methodological advantage over the comparison papers in Part 31, all of which compare against baselines they reimplemented with unstated fidelity.

PART 3 — COMPLETE SYSTEM ARCHITECTURE

3.1 The round structure

Everything happens in **rounds**. One round is:

- | | |
|------------------|---|
| 1. mobility step | vehicles move for `round_duration` seconds |
| 2. CH election | protocol selects the head set |
| 3. setup phase | ADV / JOIN / schedule, charged (our addition) |
| 4. steady state | `packets_per_round` = 22 TDMA frames of data |
| 5. trust update | watchdog observations folded into reputation |

Within a round there is **no time model below the frame**. There is no channel; transmissions never collide. This bounds what can honestly be claimed (Part 9).

3.2 Normal traffic (class 0) — the full path

stage	who	input	processing	output	why
sensing	every vehicle	—	generates <code>packets_per_round</code> readings/round	22 readings	the workload
CH election	the protocol	positions, velocities, energies, trust scores	Cuckoo Search over candidate head sets	10 heads (<code>ch_percent = 0.1</code>)	the thing being optimised
association	members	head positions	join nearest head within <code>tx_range</code>	membership + slot	our assumption; paper silent
orphaning	members with no head in range	—	fall back to direct-to-RSU	orphan set	the paper's isolated-vehicle rule
setup	heads + members	head set	ADV broadcast, JOIN, schedule broadcast, all charged at <code>ctrl_bits = 200</code>	energy debit	paper charges nothing; re-electing appears free otherwise
member TX	members	reading	<code>tx_energy(k, d_member) × 22</code>	reading at head	—
head RX	heads	member readings	<code>rx_energy(k, n_members) × 22</code>	—	—
fusion	heads	n+1 readings	<code>aggregation_energy(k, n+1) × 22</code>	one fused packet	the energy saving
head TX	heads	fused packet + MAC	<code>tx_energy(k + mac_bits, d_rsu) × 22</code>	packet at RSU	—
orphan TX	orphans	reading	<code>tx_energy(k, d_rsu) × 22</code>	packet at RSU	—

Cryptographic data carried: an **aggregate MAC** (`mac_bits = 128` , a HMAC-SHA256 tag truncated to 128 bits) under the head↔RSU key. It authenticates **the head**, not the members — see Part 19.

3.3 Emergency traffic (class 2) — where it diverges

An emergency vehicle (`ems_frac` of the fleet) marks `ems_msgs_per_round = 4` of its 22 readings as class 2. **Class-2 messages are a subset of the readings a node already sends, never extra ones** — a vehicle's radio budget does not grow because its traffic is urgent. This keeps the accounting honest and is why `ems_frac = 0` reproduces every pre-Module-5 result exactly.

stage	processing	why it differs
assertion	vehicle marks the message class 2, signs it, attaches a certificate reference	the claim must be attributable
admission	head decides whether to grant a reserved slot	this is the security decision
reservation	<code>emergency_slots = 1</code> slot per cluster per frame → capacity = $1 \times 22 = 22$ class-2 messages per cluster per round	a finite contended resource
arbitration	if admitted demand > capacity, served proportionally	without authentication the head cannot prefer the genuine claim
bypass	served messages are not fused — forwarded verbatim, individually	fusion would destroy both the deadline and the signature
energy	head refunds the aggregation it would have paid, then pays <code>tx_energy(k + sig_bits, d_rsu)</code> per message	the price of priority
fallback	demand above capacity reverts to ordinary aggregated service	and misses its deadline

Cryptographic data carried: an **ECDSA P-256 signature** plus an 8-byte certificate digest (`sig_bits = 576` total). Because the message is never transformed, that signature survives to the RSU — giving **end-to-end** authentication that the fused path cannot provide.

3.4 The three admission regimes

This is the core experiment of the project.

regime	what the head admits	config
none	every class-2 claim; genuine and false demand share the reservation proportionally	default
trust	claims from nodes the trust engine has not distrusted	<code>priority_trust_gate = True</code>
auth	only demand from vehicles holding a TA-issued EMS role attribute	<code>require_ems_auth = True</code>

3.5 Trust subsystem

Three layers, each answering an attack the previous cannot:

1. **Direct trust.** Members watchdog their head — did the aggregate reach the RSU? Evidence accumulates as $\text{Beta}(\alpha, \beta)$; trust is the posterior mean with Laplace smoothing, so an unobserved node sits at 0.5. Catches blackhole and greyhole.
2. **Decay with asymmetric penalty.** A failure counts `trust_penalty = 3` successes; evidence ages by `trust_decay = 0.9` **per observation, not per round**, and only for nodes actually observed. Without the asymmetry an on-off attacker farms trust in quiet phases and spends it in active ones.
3. **Credibility-weighted recommendations.** Reports are fused at the RSU weighted by how far each reporter has historically deviated from consensus. Bad-mouthers and ballot-stuffers accumulate deviation and lose weight.

Trust enters CH election **twice**: as a fitness term (`w_trust = 0.1`) and as a hard eligibility gate (`trust_gate = 0.4`). Module 5 adds a third lever — the priority-assertion-rate feature and the slot-grant gate.

3.6 Formal verification layer

Four Scyther models covering registration, the slot grant, the three data-path message types, and one deliberate negative result. Detail in Parts 17–20.

PART 4 — CODEBASE ARCHITECTURE

3,867 lines of Python across 24 files, plus 5 Scyther models.

4.1 Model core (the simulator)

file	lines	role	key contents
src/config.py	225	configuration	single <code>Config</code> dataclass, ~70 fields; derived <code>d0</code> , <code>n_ch</code> . Every default is commented with <i>why</i> , including the ones deliberately disabled
src/energy.py	28	baseline	<code>tx_energy</code> , <code>rx_energy</code> , <code>aggregation_energy</code> — Eqs. (9)–(11) verbatim, shared by every protocol so all are charged identically
src/network.py	445	model core	<code>Network</code> : positions, velocities, energy, RSU placement, <code>assign_members</code> , <code>run_setup_phase</code> , <code>run_steady_state</code> , <code>_priority_phase</code> , <code>_log_emerg</code>
src/mobility.py	163	model	<code>StaticMobility</code> (reproduces the paper), <code>HighwayMobility</code> , <code>UrbanGridMobility</code> ; <code>build()</code> dispatches on <code>cfg.scenario</code>
src/metrics.py	111	model	Su & Zhang link-expiration time, <code>cluster_balance</code> , <code>weighted_percentile</code> , <code>weighted_mean</code> , <code>deadline_miss</code>
src/traffic.py	128	security/model	<code>TrafficModel</code> : EMS designation, <code>demand</code> , <code>demand_authorised</code> , token bucket, per-round assertion record
src/trust.py	183	security	<code>TrustEngine</code> : Beta evidence, credibility, <code>observe</code> , <code>observe_priority</code>
src/attacks.py	113	security	<code>Adversary</code> : six behaviours, <code>drop_prob</code> , <code>falsifies_reports</code> , <code>asserts_false_priority</code>

The two functions that matter most

`Network.run_steady_state(ch_idx, adversary)` — charges one round of TDMA. Members transmit; orphans go direct; heads receive, fuse and forward. Returns the per-round record the trust engine consumes (`ch`, `members`, `slot`, `forwarded`, `orphans`). A dropping head still pays to *receive* — it cannot avoid that without leaving the TDMA schedule and revealing itself — but skips the forward transmit in proportion to what it drops. **That asymmetry is the origin of the Module 2 energy free-ride finding.**

`Network._priority_phase(...)` — the whole Module 5 mechanism: computes class-2 demand, applies the admission regime, arbitrates the finite reservation proportionally, records delay for both classes, refunds aggregation for bypassed readings, charges per-message signed forwards, and attributes the energy bill between genuine and false demand. Returns early when `self.traffic` is `None`, which is exactly why the regression gate holds.

4.2 Protocols

file	role	notes
src/protocols/csgd_net.py	baseline	faithful CSGD-NET. <code>levy()</code> implements Mantegna's algorithm. <code>fitness()</code> implements Eq. 9 with <code>fitness_mode</code> selecting the reading. <code>_walk()</code> applies continuous CS operators to a discrete node-selection problem by snapping to the nearest alive node
src/protocols/chirp.py	ours	subclasses <code>CSGDNet</code> ; overrides only <code>fitness()</code> , <code>_pool()</code> and <code>select_ch()</code> . Six-term objective; hard trust gate; CH cooldown
src/protocols/leach.py	baseline	LEACH self-election threshold $T(n)$; LEACH-C centralised, above-average-energy pool + 60-swap simulated-annealing-lite minimising Σd^2
src/protocols/heed.py	baseline	Younis & Fahmy; $CH_prob = \max(C_prob \cdot E_res / E_max, p_min)$, doubling until 1
src/protocols/swarm.py	baseline	PSO and GA over the Latiff et al. clustering cost $f = \beta \cdot f1 + (1-\beta) \cdot f2$. Note the docstring's own observation: this cost already contains the intra-cluster term Eq. 9 omits, which is why PSO and GA hold more energy than CSGD-NET late in the run

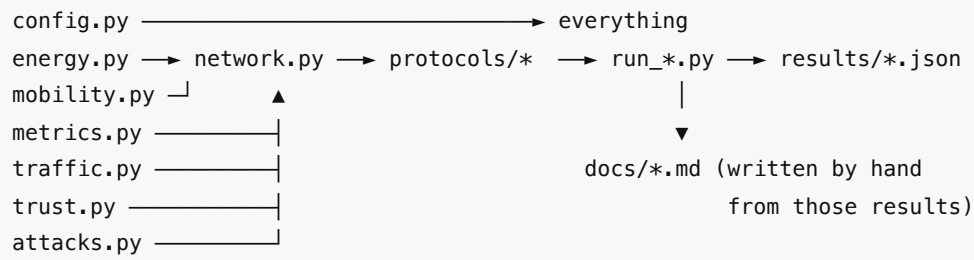
4.3 Experiment drivers

file	experiment	outputs
src/run_baseline.py	Table 4 reproduction, 6 protocols	results/baseline.json
src/run_module1.py	CH election; defines SCENARIOS (urban/highway) used by everything else	results/module1_{scen}.json , module1_ablation_{scen}.json
src/run_module2.py	trust under attack; <code>--ablate</code> for credibility on/off	results/module2_urban.json
src/run_priority.py	Module 5: <code>--bypass-cost</code> , default defence sweep, <code>--sweep-greed</code>	results/priority_{bypass,defence,greed}_{scen}.json

4.4 Verification and tooling

file	role	what it establishes
src/regression_gate.py	tooling — critical	fingerprints 55 configurations to the float bit; see Part 6
src/verify_table4.py	verification	the three-argument proof that Table 4's LEACH-C/CSGD-NET columns are unreachable
src/verify.py , src/verify_fit.py	verification	the original reproducibility audit; frame-count fitting
src/diagnose_fnd.py	verification	traces the identity and history of the first node to die, rather than guessing from aggregates
src/crypto_bench.py	verification	Module 3b: sizes from real keys, timings from openssl, energy sensitivity sweep
formal/run_scyther.py	verification	runs the 5 models, compares against the expected set, exits non-zero if anything moves

4.5 Dependency direction



`config.py` has **no imports from the project** — it is a leaf, which is what makes the regression gate able to vary one field at a time safely.

PART 5 — MODULE-BY-MODULE HISTORY

Module 1 — mobility-aware, multi-metric CH election

Original objective

Remove the static-topology assumption, and replace Eq. 9's two-term objective with one that includes the intra-cluster distance term it omits.

Initial hypothesis

A richer objective — energy, RSU distance, intra-cluster compactness, link stability, cluster balance, trust — would improve lifetime and reduce energy per delivered reading.

Implementation

- `mobility.py`: highway (1 km, 6-lane bidirectional) and urban Manhattan grid (600 m, 200 m blocks, turns at intersections). Self-contained, no SUMO dependency. `static` reproduces the paper exactly.
- `metrics.py`: Su & Zhang link expiration time, cluster balance.
- `chirp.py`: six-term fitness, stability-triggered re-clustering, CH cooldown.
- `network.py`: velocities, multi-RSU placement, **setup-phase energy**.

Two model corrections made along the way

- Setup-phase energy**. The base paper charges nothing for re-clustering, so electing every round appears free. Each election floods an ADV from every head and a JOIN from every member. `run_setup_phase()` now charges it.
- Fitness scored in energy, not distance**. The radio model has a knee at $d_0 = 87.7$ m. A head 150 m from an RSU is $1.7\times$ farther than one at 88 m but costs $\sim 9\times$ as much to reach. A distance-linear term cannot see that, so the RSU and intra terms score `tx_energy(d)` directly.

Configuration

```
w_energy 0.30, w_rsu 0.10, w_intra 0.35, w_let 0.05, w_balance 0.10, w_trust 0.10, w_survival 0.00,  
ch_cooldown 3, recluster_max_rounds 1, assoc_let_weight 0.0, energy_stat "mean"
```

Results — highway, 20 seeds (`results/module1_highway.json`)

protocol	FND	HND	LND	mJ/reading	orphan %	intra m
LEACH	19.6	33.1	44.5	0.6936	25.1	34.1
LEACH-C	25.2	31.8	41.1	0.7189	0.9	24.1
HEED	27.4	35.0	38.9	0.6577	46.8	36.4
PSO	15.5	31.9	46.0	0.7164	0.2	25.8
GA	13.8	31.2	47.0	0.7295	1.0	27.7
CSGD-NET	15.8	30.8	40.6	0.7509	10.0	38.8
CHIRP	23.0	31.8	39.9	0.7190	0.2	27.4

CHIRP vs CSGD-NET, Mann-Whitney U, all five significant: FND **+46.0 %** ($p < 0.0001$), HND +3.4 %, energy/reading −4.2 %, orphan rate −98.3 %, intra distance −29.3 %.

CHIRP is not the best FND overall — HEED (27.4) and LEACH-C (25.2) beat it, buying that with 46.8 % and 0.9 % orphan rates respectively. `docs/module1.md` says so explicitly.

Results — urban, 20 seeds (from docs/module1.md ; see the data-integrity note below)

protocol	FND	HND	LND	mJ/reading	orphan %	intra m
LEACH	8.9	14.3	20.9	1.5519	37.6	61.0
LEACH-C	9.9	18.1	26.5	1.2690	6.5	45.8
HEED	8.0	13.2	18.1	1.6988	45.4	64.5
PSO	7.7	17.3	26.6	1.3106	6.0	51.9
GA	7.5	16.2	25.8	1.3795	9.3	53.0
CSGD-NET	9.4	15.4	20.8	1.4646	21.9	60.5
CHIRP	9.4	17.6	24.2	1.2938	3.8	55.6

HND +14.6 %, energy/reading −11.7 %, orphan −82.7 %, intra −8.1 %, all $p < 0.0001$. FND 9.45 for both, $p = 0.54$ — statistically indistinguishable.

DATA-INTEGRITY WARNING. results/module1_urban.json on disk does **not** match this table. The file contains a **3-seed smoke test** (LEACH 8.7/14.7/21.0, CHIRP 9.3/17.7/24.0) that was run to check the runner still executed during the reproduction-gap work, and was then committed. The **20-seed table in docs/module1.md is the authoritative research result.** results/module1_highway.json is *not* affected — it matches its doc exactly. Action item: re-run `python3 src/run_module1.py --scenario urban --seeds 20` to restore consistency. Not done during this extraction, to avoid modifying the research.

Ablation — highway, 10 seeds (results/module1_ablation_highway.json)

dropped	FND	HND	mJ/reading	orphan %	intra m
none (full)	16.2	31.5	0.7278	0.5	28.6
energy	10.3	31.8	0.7177	0.3	29.7
rsu	16.4	31.5	0.7287	0.5	27.7
intra	15.6	30.6	0.7441	0.6	32.4
let	16.2	31.4	0.7310	0.7	28.5
balance	15.8	31.5	0.7299	0.4	29.3

Only **energy** (−37 % FND when dropped) and **intra** (+2.2 % energy, +13 % intra distance when dropped) are load-bearing. Weights were reallocated on that evidence — LET 0.25 → 0.05, into energy and intra — rather than chosen by hand. **The intra term is the one Eq. 9 omits, and the ablation confirms it is the second most important term in the objective.**

Unexpected results — three of them, all negative

- Stability-triggered re-clustering backfired.** Holding heads across rounds saves setup traffic but concentrates the head role. Diagnosis via `diagnose_fnd.py`: serving as head costs **88.00 mJ/round** in urban against **0.2833 mJ** for one re-election — **311×**. Raising the ceiling from 1 to 5 cost 26 % of urban FND and 35 % of highway FND to save a negligible amount of control energy. **Default recluster_max_rounds = 1** — re-elect every round, plus a LEACH-style cooldown.
- LET-aware member association backfired**, monotonically. Sweeping α from 0 (nearest) to 1 (longest-lived link) on the highway: FND 16.1 → 14.1, mJ/reading 0.7275 → 0.7916, intra 28.3 → 37.7 m, while re-clustering barely moved (0.78 → 0.79). At 1 node per 10 m there is always a nearby same-direction candidate, so biasing toward link lifetime only buys distance. **Default $\alpha = 0$.**
- The survival term backfired**, monotonically. A term predicting each head's round cost and rewarding the weakest head's remaining rounds made urban FND worse (8.5 → 7.0 as the weight went 0 → 0.65) because orphans do not count toward a head's predicted cost, so the optimiser raised minimum survival **by orphaning members** — orphan rate climbed 3.9 % → 6.0 %. **Default 0.**

Interpretation

CHIRP's first casualty is a cluster head 85 % of the time, having served a 10.6-member cluster (vs CSGD-NET's 6.1). Better coverage *causes* it: CSGD-NET orphans 21.9 % of nodes and orphans pay their own direct-to-RSU cost; CHIRP absorbs them into clusters, moving that cost onto the heads, and receive energy scales linearly with cluster size.

Final conclusion

What carries Module 1 is narrower and better supported than six helpful terms: the intra-cluster term Eq. 9 omits, energy-scored rather than distance-scored costs, and cluster-head rotation via a cooldown. Three initial design decisions were wrong and the measurements say so.

Files

```
src/mobility.py, src/metrics.py, src/protocols/chirp.py, src/network.py, src/run_module1.py,
src/diagnose_fnd.py, docs/module1.md
```

Commands

```
python3 src/run_module1.py --scenario highway --seeds 20
python3 src/run_module1.py --scenario urban --seeds 20 --rounds 150
python3 src/run_module1.py --scenario highway --seeds 10 --ablate
python3 src/run_module1.py --scenario highway --seeds 8 --sweep-assoc
python3 src/diagnose_fnd.py
```

Status

COMPLETE. One data-integrity action item (regenerate `module1_urban.json`).

Module 2 — trust engine and Layer-A attacks

Original objective

Add the defence the base paper has no equivalent of. Eq. 9 scores a candidate head on residual energy and distance to the RSU; both are self-reported and neither is a behavioural observation, so an attacker claiming a full battery near an RSU wins the head role and drops whatever it likes.

Initial hypothesis

Beta-reputation trust feeding the Cuckoo fitness plus a hard eligibility gate will keep malicious nodes out of the head role and raise PDR under attack.

Implementation

- `attacks.py` — Layer-A adversary: blackhole, greyhole, on-off, bad-mouthing, ballot-stuffing. **Internal** attackers holding valid identities.
- `trust.py` — three-layer Beta reputation (Part 3.5).
- `network.py` — heads forward only a fraction of what they received; PDR measured as readings delivered over readings generated.
- `chirp.py` — trust as a fitness term **and** a hard eligibility gate.

Configuration

```
watchdog_p 0.85, watchdog_err 0.05, obs_per_round 22, trust_decay 0.90, decay_unobserved False,
trust_penalty 3.0, trust_gate 0.40, cred_sharpness 4.0, trust_warmup 8, eval_alive_frac 0.5,
greyhole_drop 0.5, onoff_period 8
```

Two subtleties worth explaining at a viva: - `obs_per_round = 22`, not 1. A member sends 22 packets to its head per round and can watchdog every one. Counting one observation per round starves the estimator — a greyhole is only elected head every few rounds, so it would take tens of rounds to cross the gate. - `decay_unobserved = False`. Ageing evidence

for *unobserved* nodes silently rehabilitates an attacker the moment the gate stops electing it — which is precisely when it should stay excluded.

Results — urban, 12 seeds, mixed attack (docs/module2.md)

attacker %	protocol	PDR	att CH %	late %	detect	FPR	TTD
10	CSGD-NET	0.968	10.0	10.3	—	—	—
10	CHIRP + trust	0.979	8.1	6.4	0.831	0.018	6.4
20	CSGD-NET	0.934	21.9	20.3	—	—	—
20	CHIRP + trust	0.957	16.9	14.8	0.815	0.033	5.8
30	CSGD-NET	0.887	32.9	35.0	—	—	—
30	CHIRP + trust	0.923	23.1	19.4	0.878	0.044	5.5
40	CSGD-NET	0.859	40.8	39.7	—	—	—
40	CHIRP + trust	0.904	33.3	29.5	0.819	0.035	5.8

PDR at 20 % attackers: **0.957 vs 0.934, +2.4 %, p = 0.0018**. At 40 %, +5.2 %. At 0 % attackers all four protocols deliver PDR 1.000 — the trust machinery costs nothing when there is nothing to defend against.

CHIRP (no trust) tracks CSGD-NET closely (0.843 vs 0.859 at 40 %). **The Module 1 objective is not itself a defence and must not be presented as one — the entire security gain comes from the trust layer.**

Same data-integrity warning as Module 1. results/module2_urban.json holds a **3-seed smoke test** (20 % CHIRP+trust PDR 0.945, detect 0.836), not the 12-seed run above. docs/module2.md is authoritative.

Credibility ablation — collusion mix, 10 seeds

attacker %	credibility	detect	FPR
20	on	0.882	0.036
20	OFF	0.913	0.053
30	on	0.767	0.070
30	OFF	0.793	0.091
40	on	0.796	0.092
40	OFF	0.710	0.178

Credibility weighting cuts the **false-positive rate** by 12–49 %, and by 49 % at 40 % attackers. Detection rate is *not* uniformly improved — at 20 % and 30 % it is slightly better without. The benefit is concentrated in false positives, and the report should say so rather than claim a uniform win.

Unexpected result — the defence gives attackers an energy free ride

Mean residual energy of attackers relative to honest nodes at end of run:

attacker %	CSGD-NET	CHIRP (no trust)	CHIRP + trust
10	1.01	1.23	1.98
20	1.11	1.47	2.11
30	1.27	1.40	2.41
40	1.21	1.47	2.43

Two effects compound. Dropping already skips the forward transmit, so attacking is cheaper than behaving even undefended (1.2–1.5×). The trust gate then *excludes* misbehaving nodes from the head role — the most expensive job in the network at 88 mJ/round against 11.6 for an ordinary member — so a **detected attacker is spared the very duty that drains honest nodes**, and ends with 2.0–2.4× their residual energy.

Interpretation

This is a real limitation of demotion-based defence. Two remedies, both belonging to a future Module 4: **revocation rather than demotion**, and **duty rebalancing**. It also means **a security defence cannot be validated on energy metrics alone** — an attacker wins on energy by construction. This directly indicts the headline metrics of the comparison papers in Part 31.

Files

```
src/attacks.py, src/trust.py, src/network.py, src/protocols/chirp.py, src/run_module2.py, docs/module2.md
```

Commands

```
python3 src/run_module2.py --scenario urban --seeds 12
python3 src/run_module2.py --scenario urban --seeds 10 --ablate
```

Status

COMPLETE. Urban JSON needs regenerating at 12 seeds.

Module 3a — Scyther formal verification

Full detail in Parts 17–20. Summary here for chronological completeness.

Original objective

Discharge, rather than assume, the property every Module 5 authorisation result rests on: *a cluster head grants a reserved emergency slot only to a vehicle holding a TA-issued EMS role attribute, and never to a replay.*

Initial hypothesis

The models would verify cleanly and the section would be conventional.

Implementation

Five `.spdl` models plus `formal/run_scyther.py`, which compares results against the expected set and exits non-zero if anything moves.

Results

Scyther v1.3.0, `--max-runs=4` : 32 claims verified, 8 failing by design.

Unexpected results — two

1. **Scyther found a real cross-protocol attack** in our aggregate design (Part 18). Fixed with domain-separation tags.
2. **Modelling the relay as a role was wrong** — it makes Nisynch require the relay to be honest, which is weaker than reality. Under Dolev-Yao the network already is the adversary, so a two-role model is correct.

Status

COMPLETE.

Module 3b — measured cryptographic cost

Full detail in Parts 21–23.

Original objective

Fill in `sig_bits` and `mac_bits`, which Module 5 had left at 0 with the note that its cost of priority was therefore a lower bound.

Initial hypothesis

Recorded in `docs/priority.md` before the measurement: filling them in "*will raise the cost of the bypass*".

Results

`sig_bits = 576`, `mac_bits = 128`. Whole crypto layer plus bypass costs **+1.63 % urban**, **+0.77 % highway**; the per-message signature is **statistically undetectable** ($p = 0.94$ highway).

Unexpected result

The prediction was wrong in an interesting way: the transmission cost is negligible, but **computation** cost is $\sim 15\times$ the radio energy of the signature it verifies — and the base paper's model charges radio only, so it is structurally incapable of showing this.

Status

COMPLETE.

Module 4 — RSU-side IDS

Status

NOT STARTED. Verified by search: no `src/run_module4.py`, no `src/ids.py`, no `docs/module4.md`, no `sklearn` import anywhere in `src/`. The only occurrence of "VeReMi" in code is a docstring in `src/attacks.py` explaining which attacks VeReMi does *not* contain. See Part 30.

Module 5 — priority traffic

Full detail in Parts 9–14.

Original objective

Give emergency vehicles a traffic class with a deadline, and find out what carrying it costs inside an aggregation protocol.

Initial hypothesis

Bypassing fusion would be expensive, and the trust engine already built in Module 2 would handle any abuse of the fast path.

Both halves were wrong

Bypassing fusion is cheap (1–2 % of network energy). And trust — despite detecting the attacker at 0.93–0.96 — changed the outcome **not at all** until it was given an enforcement path, and even then cannot stop a restrained attacker.

Status

COMPLETE, both scenarios, with crypto charged.

PART 6 — THE REGRESSION METHODOLOGY

6.1 Why it exists

Module 5 adds traffic classes, a delay model, a reservation mechanism and per-message crypto to the same `run_steady_state` that produces every Module 1 and Module 2 result. Without a guard, a small accounting change in the priority path would silently shift the energy numbers underneath the Table 4 reproduction and the entire protocol comparison — and nobody would notice, because those results live in documents written weeks earlier.

The regression gate is what makes it safe to keep extending the simulator.

6.2 What it protects

The bit-exact behaviour of the baseline, Module 1 and Module 2 experiments — i.e. every result the project's comparisons rest on.

6.3 How it works

`src/regression_gate.py` runs a fixed matrix of configurations on fixed seeds and fingerprints each one with **SHA-256 over the raw float64 bytes** of the state:

```
def fingerprint(a):  
    return hashlib.sha256(np.ascontiguousarray(a, dtype=np.float64).tobytes()).hexdigest()[:16]
```

Per configuration it records:

field	why
<code>energy_fp</code>	SHA-256 of the entire 100-node energy vector — catches any change a rounded mean would hide
<code>residual</code>	mean residual energy, 12 dp
<code>n_alive</code> , <code>rounds_run</code>	lifetime behaviour
<code>packets_to_rsu</code> , <code>readings_generated</code> , <code>readings_delivered</code>	PDR counters
<code>orphan_fp</code> , <code>intra_fp</code>	SHA-256 of the full per-round orphan-rate and intra-distance traces
<code>trust_fp</code>	SHA-256 of the full trust-score vector (trust runs only)

6.4 The 55 configurations

group	matrix	count
static (the paper's reproduction setting)	7 protocols × 3 seeds	21
highway	7 protocols × 2 seeds	14
urban	7 protocols × 2 seeds	14
trust engine under mixed attack	3 attacker fractions (0.0/0.2/0.4) × 2 seeds	6
	total	55

They matter because they span **every code path that Module 5 could disturb**: all three mobility models, all seven protocols (so both the Cuckoo engine and the non-Cuckoo baselines), the adversary, and the trust engine. A change that affected only urban, or only CHIRP, or only the trust path, would still be caught.

6.5 The invariant, stated precisely

With `ems_frac = 0` and `sig_bits = mac_bits = 0` — that is, with no emergency vehicles and no cryptographic overhead — **every pre-Module-5 result must be bit-identical**.

Structurally this holds because `Network._priority_phase()` returns immediately when `self.traffic` is `None`, and the traffic model is attached by the *runner*, never constructed inside `Network`. That last detail is deliberate: constructing it inside `Network` would consume draws from `self.rng` and shift every mobility trace.

6.6 Why this protects scientific validity

Module 5's headline claims are **comparative** — "the bypass costs +1.8 %", "authorisation eliminates a 44 % deadline-miss". Each is a difference against a baseline. If the baseline drifted while the feature was being built, every such difference would be measuring two changes at once and no amount of statistics would separate them. The gate turns "we don't think we broke anything" into "55 configurations are bit-identical, here is the diff".

It also caught the crypto default change in Module 3b: setting `sig_bits = 576` and `mac_bits = 128` left all 55 identical, confirming that crypto genuinely only applies when a traffic model is attached.

6.7 When to run it

Before and after any change to `src/`.

```
python3 src/regression_gate.py /tmp/now.json
python3 src/regression_gate.py --diff results/regression_reference.json /tmp/now.json
```

Non-empty diff + unintended change = the reproduction is broken. The reference is `results/regression_reference.json` (16,751 bytes, 55 keys), committed.

PART 7 — DETERMINISM

7.1 Seed handling

Every stochastic component gets its **own** generator, derived from one integer seed by a fixed offset:

generator	seed	governs
Network	seed	node placement, mobility trace, watchdog observation noise
protocol	seed + 10_000	Cuckoo/LEACH/HEED/PSO/GA randomness
Adversary	seed + 555	which nodes attack and with what behaviour
TrafficModel	seed + 999	which vehicles are emergency vehicles

Why separate streams matter: the same seed gives the *same road*, *same speeds*, *same node positions* regardless of which protocol is running. Every protocol is therefore evaluated on identical mobility traces, so a difference between protocols cannot be a difference in the scenario they were handed.

`np.random.default_rng` (PCG64) is used throughout — no global `np.random` state anywhere, so execution order cannot leak between components.

7.2 What "55/55 identical" means

It means: for all 55 configurations, **every recorded field matched exactly**, including four SHA-256 fingerprints per configuration computed over raw float64 bytes. Not "the means agreed to 4 decimal places" — the 100-element energy vector hashed to the same 16 hex characters.

This was verified by running the gate twice on unmodified code before any Module 5 work began, then after each of steps 1–5, after the Module 3a work, and after the Module 3b config change. It has passed at every checkpoint.

7.3 What determinism does *not* claim

- It does not make the results *correct*, only *stable*.
 - It does not cover the priority experiments themselves — those are gated by re-running `run_priority.py` and diffing the JSON, which was done for `priority_defence_urban.json` (18/18 keys identical) after the Module 3a work.
 - Wall-clock timings in `crypto_bench.py` are inherently non-deterministic; that script is a measurement, not a reproduction, and reports which machine produced the numbers.
-

PART 8 — MODULE 1 IN DEPTH

8.1 The protocols compared

protocol	family	CH selection
LEACH	distributed, randomised	each node self-elects with threshold $T(n) = p / (1 - p \cdot (r \bmod 1/p))$, so every node is head once per $1/p$ rounds
LEACH-C	centralised	RSU knows all energies and positions; picks from the above-average-energy pool the set minimising $\Sigma(\text{member} \rightarrow \text{CH})^2$, by 60 swap iterations
HEED	distributed, iterative	$\text{CH_prob} = \max(\text{C_prob} \cdot E_{\text{res}}/E_{\text{max}}, p_{\text{min}})$, doubling each iteration until 1; intra-cluster cost as tie-break
PSO / GA	metaheuristic	minimise Latiff et al. cost $f = \beta \cdot f1 + (1-\beta) \cdot f2$, $f1$ = worst-cluster mean member distance, $f2$ = network energy / head energy
CSGD-NET	metaheuristic (base paper)	Cuckoo Search, Eq. 9 $f = E / (E0 \cdot \text{dist}(\text{CH}, \text{RSU}))$, maximised
CHIRP	metaheuristic (ours)	same Cuckoo engine, six-term objective, hard trust gate, cooldown

Important observation, recorded in `src/protocols/swarm.py`'s own docstring: the PSO/GA cost *already contains* the intra-cluster term Eq. 9 omits. That is why PSO and GA hold more energy than CSGD-NET late in the run in our reproduction — the base paper's own comparison protocols already have the term the base paper's objective is missing.

8.2 CHIRP's objective

```
f(S) = w_e·E + w_r·(1 - D_rsu) + w_i·(1 - D_intra) + w_l·LET + w_b·B + w_t·T
all six terms normalised to [0,1] and maximised,
then multiplied by `coverage` = members_in_range / (members + orphans)
```

- **E** — mean residual energy ratio of the heads (`energy_stat = "mean"`)
- **D_rsu** — mean head→nearest-RSU cost, **scored as `tx_energy`, not distance**, normalised against `tx_energy(tx_range)`
- **D_intra** — mean member→head cost, likewise in energy. **The term Eq. 9 omits**
- **LET** — Su & Zhang link expiration time, normalised against the re-clustering horizon
- **B** — cluster-size balance, $1/(1 + \text{CV of cluster sizes})$
- **T** — mean trust of the heads (1.0 when no trust engine attached)

The `coverage` multiplier penalises orphaning smoothly rather than by a hard constraint — an election that strands members helps nobody.

8.3 Eligibility: the pool

`CHIRP._pool()` narrows candidates twice: 1. **Trust gate** — nodes below `trust_gate = 0.4` cannot be elected at all, falling back to the ungated pool if that would leave fewer than `n_ch` candidates. 2. **Cooldown** — a node cannot be head again within `ch_cooldown = 3` rounds, falling back to the full pool rather than starve the election.

8.4 The radio model and the d0 knee

```
d0 = sqrt(ε_fs / ε_mp) = sqrt(1e-11 / 1.3e-15) = 87.71 m
```

Below 87.71 m the amplifier term is $\epsilon_{\text{fs}} \cdot d^2$; above it, $\epsilon_{\text{mp}} \cdot d^4$. Consequences that run through the whole project:

- In the paper's **50 × 50 m** field (diagonal 70.7 m) the d^4 branch **never fires**. Every link is free-space. This is why protocols cannot separate on energy there (Part 16).

- In **urban** (600 m grid, 9 RSUs, `tx_range` 150 m) many head→RSU links exceed 87.7 m, so they are charged d^4 and are disproportionately expensive.
- In **highway** (1 km, 5 RSUs, `tx_range` 100 m) heads are almost always inside the knee.

That single difference explains the RSU-density finding in Part 15.

`_place_rsus()` exists because of this: placing RSUs in a line down the middle of a 600×600 m grid leaves corners over 300 m from the nearest one, which at d^4 drains those nodes in a single round — that is what made every early urban run report FND = 1.

8.5 Orphans — the mechanism that drives the headline finding

A node with **no cluster head within `tx_range`** is an orphan and transmits **directly to the RSU**, per the base paper's isolated-vehicle rule.

Per-round cost by role, urban, E0 = 500 mJ (`src/diagnose_fnd.py`):

role	cost/round	rounds to death
cluster head (8 members)	88.00 mJ	5.7
orphan (direct to RSU)	25.34 mJ	19.7
ordinary member	11.61 mJ	43.0
one re-election (setup)	0.2833 mJ	—

Two things follow. Serving as head costs **311×** what one re-election costs — which is why holding heads across rounds was a mistake. And an orphan is expensive **for itself** but cheap for the network's heads.

8.6 THE MAJOR FINDING — coverage versus emergency latency

Measured before any bypass existed, so every reading is aggregated. 6 seeds, 5 % EMS, deadline 4 slots.

protocol	urban orphan %	delay	dl-miss	highway orphan %	delay	dl-miss
HEED	46.8	4.22	0.273	48.0	5.24	0.387
LEACH	38.5	6.91	0.602	31.2	8.15	0.609
CSGD-NET	19.3	7.70	0.687	10.1	10.14	0.820
GA	9.2	8.64	0.782	1.1	10.44	0.898
PSO	6.0	9.08	0.813	0.2	10.43	0.924
LEACH-C	5.9	9.16	0.888	1.1	10.59	0.953
CHIRP	3.1	9.68	0.874	0.1	10.23	0.958

Deadline-miss tracks orphan rate almost perfectly inversely, in both scenarios, across seven protocols.

Why fewer orphans means worse emergency latency

Because **orphans skip fusion**. An orphan sends straight to the RSU and waits for nobody — delay 1 slot. A clustered member's reading cannot leave until its head has heard *every* member, because the head is fusing them all into one packet — delay $\approx$ cluster size + 1.

So a protocol that is good at coverage: - absorbs would-be orphans into clusters (good for energy, good for PDR), - thereby makes clusters **larger**, - thereby makes the frame **longer**, - thereby makes every reading in that cluster **later**, - and removes the accidental fast path that orphaning provided.

HEED, which strands nearly half the network, has the **best** emergency latency in the comparison *precisely because it is the worst at clustering*.

Why this is a negative result for our own work

CHIRP's headline Module 1 achievement is an 82.7 % urban / 98.3 % highway reduction in orphan rate. On this metric that achievement is a **loss**. The effect is sharper on the highway (0.958 vs urban's 0.874) because highway CHIRP orphans 0.1 % of nodes — there is no accidental fast path left at all.

It is the fourth published negative result in the project, and it is what motivates Module 5: if you want both coverage *and* deadlines, you need an explicit fast path, because clustering quality alone will always trade one against the other.

PART 9 — THE DELAY MODEL

9.1 Why it was introduced

The base paper reports energy and nothing else. "Emergency traffic arrives late" is not a claim you can make without a latency metric, and the project had none.

9.2 The unit — and why not milliseconds

Delay is measured in TDMA slots. Never milliseconds. The simulator has no channel and no time model below the round, so a millisecond figure would be invented. But a TDMA schedule **is** a real, finite, contended resource in the base paper's own model, and the delay it imposes is exactly computable.

The metric is reported as "*normalised scheduling delay (TDMA slots)*".

9.3 The three paths

aggregated	= cluster_size + 1	member slots, then the head's forward
bypassed	= 2	reserved slot, then an immediate forward
orphan	= 1	direct to the RSU, nothing to wait for

The key property: **aggregated delay scales with cluster size; bypassed delay is constant.** A member's own slot position does not matter — everyone waits for the frame to close, because the head cannot fuse until it has heard everyone.

9.4 Deadline, percentiles, and what counts as a miss

- `deadline_slots = 4.0`. Bypassed traffic (2 slots) always makes it; aggregated traffic makes it only in clusters of ≤ 3 members.
- The deadline applies to class-2 only.** Normal telemetry is delay-tolerant by definition — that is why it is aggregatable — so reporting a deadline-miss for it would be meaningless. Class-0 gets a delay *distribution* instead.
- deadline_miss counts a dropped message as a miss.** For an ambulance the distinction between "arrived late" and "never arrived" is not worth drawing, and keeping them separate would let a defence look good on latency while quietly losing the message. Pure latency is reported separately as p95 over *delivered* readings.

9.5 Bookkeeping

Delay is recorded **per cluster per round** as `(slots, n_readings)` weighted samples — roughly 10 entries a round instead of ~ 2000 — and `metrics.weighted_percentile` honours the weights, so the percentiles are identical to per-reading recording. Genuine-EMS traffic is tracked in a separate series (`delay_emerg_gen`) from all class-2 traffic, so the *safety* metric measures what a real ambulance experienced rather than what everything claiming to be one did.

9.6 The prediction, and its confirmation

The inversion in Part 8.6 was **predicted before it was measured**, from the mechanism: CHIRP cuts the orphan rate, orphans are the low-delay path, therefore CHIRP should be worse on emergency latency. It was flagged as an expected outcome so it would not be mistaken for a bug when it appeared. The measurement confirmed it across seven protocols and two scenarios.

PART 10 — PRIORITY BYPASS

10.1 Why

Fusion is what makes emergency traffic late (Part 8.6). The only way to decouple emergency delay from cluster size is to **not fuse it**.

10.2 How it works

Per cluster, per round:

1. **Demand** — class-2 messages offered by members and the head itself.
2. **Admission** — depends on regime (none / trust / auth).
3. **Capacity** — $\text{emergency_slots} \times \text{packets_per_round} = 1 \times 22 = \mathbf{22 \text{ class-2 messages per cluster per round}}$.
4. **Serve** — $\text{served} = \min(\text{admitted}, \text{capacity})$; if oversubscribed, shares are **proportional**.
5. **Bypass accounting** — for each served message the head: - is **refunded** the aggregation it would have paid ($\text{aggregation_energy}$), - **pays** $\text{tx_energy}(k + \text{sig_bits}, d_{\text{rsu}})$ to forward it verbatim, - and if *every* reading in the cluster was bypassed, is refunded the fused packet too.
6. **Fallback** — demand above capacity reverts to aggregated service and misses its deadline.
7. **MAC** — the fused class-0 packet is charged mac_bits extra, **every frame**, whether or not any emergency traffic exists.

Normal traffic is untouched: still fused, still one packet per frame.

10.3 Results — 12 seeds, 5 % EMS, no attackers, crypto charged

Source: `results/priority_bypass_urban.json`, `priority_bypass_highway.json`

Urban (600 m grid, 9 RSUs, `tx_range` 150 m)

protocol	bypass	mJ/reading	EMS delay	EMS p95	dl-miss
CSGD-NET	off	1.4974	7.67	16.2	0.668
CSGD-NET	on	1.5148 (+1.2 %)	1.75	2.0	0.000
CHIRP	off	1.3164	9.99	15.8	0.913
CHIRP	on	1.3408 (+1.8 %)	1.95	2.0	0.000

Highway (1 km 6-lane, 5 RSUs, `tx_range` 100 m)

protocol	bypass	mJ/reading	EMS delay	EMS p95	dl-miss
CSGD-NET	off	0.7655	10.58	20.7	0.856
CSGD-NET	on	0.7645 (−0.1 %, noise)	1.91	2.0	0.000
CHIRP	off	0.7285	10.57	15.7	0.988
CHIRP	on	0.7323 (+0.5 %)	2.00	2.0	0.000

Deadline misses go to zero in both scenarios for under 2 % of network energy. p95 collapses from 15.7–20.7 slots to exactly 2.0.

10.4 How to read those percentages — a caveat that matters

The aggregate MAC is charged in both arms. Turning the bypass off does not turn the crypto layer off, so these ratios mix two effects, and the CSGD-NET highway −0.1 % is noise rather than a saving. Module 3b decomposes them properly with paired tests (Part 22): the honest figure is **+1.63 % urban**, **+0.77 % highway** for the whole crypto layer plus bypass.

10.5 The reservation is comfortably sufficient — until attacked

With genuine ambulances only: 5 EMS vehicles $\times$ 4 messages = 20 class-2 messages per round across ~ 10 clusters, against a capacity of 22 **per cluster**. It never saturates. That is by design and it sets up Part 11 — **only an attacker can exhaust it**.

PART 11 — PRIORITY AS A DENIAL-OF-SERVICE SURFACE

11.1 The attack

`falsepriority` — a sixth adversary behaviour in `src/attacks.py`. The attacker **drops nothing**. It attacks a *resource*.

- Emergency slots are reserved and finite.
- A cluster head with no way to authenticate a claim must share them.
- A genuine ambulance asserts on `ems_msgs_per_round = 4` of its 22 readings.
- A false-priority attacker asserts on `falsepriority_rate × 22` — at the default greed of 1.00, **all 22**.

One attacker in a cluster exactly saturates the reservation (22 = capacity).

11.2 The asymmetry, stated precisely

At 5 % attackers: 5 attackers × 22 = **110** class-2 assertions per round against 5 ambulances × 4 = **20**. The attacker population is the same size as the ambulance population, but its demand is **5.5× larger**, because each attacker pushes 5.5× harder. **The asymmetry is not in the number of attackers — it is in how hard each one pushes**, and no amount of counting vehicles reveals it.

11.3 Cost to attacker versus cost to victim

This is the inverse of the Module 2 free ride:

	Module 2 dropper	Module 5 false-priority
attacker energy	saves (skips forwards it owed)	spends nothing extra
honest energy	unchanged	head pays to forward each fake-urgent message verbatim
what it buys	2.0–2.4× residual energy	an ambulance's deadline

The Module 2 attacker **banks** the free ride. The Module 5 attacker **externalises** it. And what it costs is not battery — it is a safety deadline.

11.4 Results — 12 seeds, CHIRP, 5 % EMS, greed 1.00, crypto charged

Source: `results/priority_defence_{urban,highway}.json`

Urban

att %	defence	EMS dl-miss	EMS delay	priority mJ	false %	net drain %	detect	FPR
0	none	0.000	1.95	560	0	0.00	—	—
0	trust	0.002	1.96	562	0	0.00	—	—
0	auth	0.000	1.94	563	0	0.00	—	—
5	none	0.110	3.07	2848	83	5.05	—	—
5	trust	0.042	2.26	972	37	0.89	0.983	0.023
5	auth	0.000	1.96	573	0	0.00	0.950	0.016
10	none	0.205	3.97	4061	90	7.78	—	—
10	trust	0.145	3.15	1090	52	1.34	0.967	0.022
10	auth	0.000	1.94	572	0	0.00	0.958	0.022
20	none	0.437	6.19	5521	95	11.26	—	—
20	trust	0.234	4.18	1650	72	2.75	0.942	0.015
20	auth	0.000	1.94	549	0	0.00	0.937	0.015
30	none	0.546	7.13	6028	97	12.42	—	—
30	trust	0.268	4.28	1743	75	3.02	0.953	0.014
30	auth	0.000	1.94	571	0	0.00	0.953	0.020
40	none	0.642	7.97	6342	97	13.25	—	—
40	trust	0.392	5.53	1921	84	3.71	0.948	0.026
40	auth	0.000	1.92	533	0	0.00	0.950	0.026

Highway

att %	defence	EMS dl-miss	EMS delay	priority mJ	false %	net drain %	detect	FPR
0	none	0.000	2.00	328	0	0.00	—	—
0	trust	0.000	2.00	324	0	0.00	—	—
0	auth	0.000	2.00	324	0	0.00	—	—
5	none	0.118	3.14	1575	82	2.67	—	—
5	trust	0.026	2.23	438	26	0.26	1.000	0.008
5	auth	0.000	2.00	315	0	0.00	1.000	0.010
10	none	0.245	4.32	2316	90	4.30	—	—
10	trust	0.131	3.11	436	34	0.36	0.992	0.009
10	auth	0.000	2.00	309	0	0.00	1.000	0.011
20	none	0.472	6.43	3073	95	6.01	—	—
20	trust	0.260	4.27	497	55	0.65	1.000	0.014
20	auth	0.000	2.00	296	0	0.00	1.000	0.007
30	none	0.595	7.50	3402	96	6.74	—	—
30	trust	0.281	4.45	785	72	1.38	0.975	0.012
30	auth	0.000	2.00	283	0	0.00	0.986	0.010
40	none	0.705	8.43	3432	97	6.89	—	—
40	trust	0.446	5.83	895	83	1.87	0.962	0.007
40	auth	0.000	1.99	274	0	0.00	0.960	0.011

11.5 What to read off these

- **At just 5 % attackers, 82–83 % of the reservation's energy already serves liars.** Identical in both scenarios, because it is a property of the traffic model rather than the road.

- **Up to 13.25 % of the entire network's energy budget** goes to forwarding falsely-urgent messages (urban, 40 %).
 - **The drain is roughly half as bad on the highway** (6.01 % vs 11.26 % at 20 %) — same reason priority is cheaper there, heads sit inside the d0 knee. RSU density mitigates the attack as well as the feature's cost.
 - **Authorisation is 0.000 at every attacker fraction in both scenarios**, and its priority-energy column stays flat (~530–573 urban, ~274–328 highway) — the false demand never gets forwarded at all.
-

PART 12 — THE TRUST SYSTEM AND "DETECTION ≠ MITIGATION"

12.1 What trust measures, and by what mechanism

Trust is a **behavioural** estimate built from watchdog observations of *forwarding*: did the head relay the aggregate to the RSU? Evidence accumulates as $\text{Beta}(\alpha, \beta)$; trust is the Laplace-smoothed posterior mean. A node below `trust_gate = 0.4` is distrusted.

Module 5 adds a second, different signal — `TrustEngine.observe_priority()` — which folds **priority-assertion rate** into the same reputation. The two signals are epistemically different, and the difference is the point:

	forwarding behaviour	priority assertion
visibility	private — only a listening member knows	public — stamped on the message
noise	observations are noisy, reporters lie	none
needs consensus?	yes — credibility weighting exists for this	no
needs a threshold?	no	yes — and that is where the attack lives

12.2 The first trust experiment, and its failure

The first version gated only **CH eligibility**, as Module 2 did.

Result at 20 % attackers: **detection 0.925, deadline-miss 0.452 undefended versus 0.463 with trust**. Detection was excellent. The outcome was *unchanged* — very slightly worse.

Why

The trust gate's only lever was cluster-head eligibility, and asserting priority does not require being a cluster head. The detector was wired to an enforcement path the attack does not traverse. Distrusting a node perfectly, and then still granting it a reserved emergency slot, accomplishes nothing.

This is the most transferable lesson in the project: a detection rate is not a security result until you name the enforcement path. A paper reporting "97 % attack detection rate" has said nothing about outcomes unless it also shows what the detection *did*.

12.3 The fix — an enforcement path

`priority_trust_gate = True`: heads refuse reserved slots to nodes the network distrusts. Implemented in `_priority_phase` by masking admitted demand with `self.trust.trusted(...)`.

With that, trust roughly **halves** the damage: urban 0.437 → 0.234 at 20 % attackers, highway 0.472 → 0.260. Net drain falls from 11.26 % → 2.75 % urban and 6.01 % → 0.65 % highway.

12.4 The cost to honest traffic

Trust gating is **statistical**, so it inherits the trust engine's false positives: an ambulance wrongly distrusted **loses its reserved slot**. This is visible at **zero attackers**, where the urban trust row shows a non-zero EMS deadline-miss (0.002 in the current run) while authorisation shows exactly 0.000.

Important honesty note. That figure moved between runs — 0.015 before the crypto re-run, 0.002 after — and crypto cannot plausibly affect a trust false-positive rate. It is seed noise at 12 seeds. **The claim is therefore directional only: a statistical gate on a safety path occasionally charges honest traffic; a cryptographic one structurally cannot.** Pinning the magnitude down would need far more seeds than the conclusion warrants.

12.5 Urban versus highway

Detection is **perfect on the highway** (1.000 at 5/10/20 % attackers) and merely excellent in urban (0.942–0.983), at roughly a third the false-positive rate. Reason: highway networks live about twice as long (FND 23 vs 9.6 rounds), so the trust engine gets roughly twice the rounds to accumulate assertion-rate evidence before the evaluation window closes at half-node-death. **Detection quality here is bounded by network lifetime, not by the detector.**

The false-positive cost likewise does not appear on the highway at all (0.000 at 0 % attackers) — the longer lifetime lets the engine resolve honest nodes.

PART 13 — THE GREED / STEALTH SWEEP

13.1 The greed parameter

`falsepriority_rate` $\in [0, 1]$ — the fraction of its own readings on which a false-priority attacker asserts class 2.

- **1.00** — asserts on all 22. Maximum damage, maximum visibility.
- **0.18** — `ems_msgs_per_round / packets_per_round` = 4/22. **Exactly the rate of a genuine ambulance.**
- The behavioural detector flags above `priority_flag_rate` = 0.50.

13.2 Results — 20 % attackers, 12 seeds

Source: `results/priority_greed_{urban,highway}.json`

greed	defence	urban dl-miss	drain %	detect	highway dl-miss	drain %	detect
0.18	none	0.013	4.48	—	0.011	2.41	—
0.18	trust	0.011	4.26	0.025	0.021	2.44	0.004
0.18	auth	0.000	0.00	0.013	0.000	0.00	0.021
0.30	none	0.067	6.76	—	0.072	3.79	—
0.30	trust	0.074	6.69	0.021	0.077	3.73	0.000
0.50	none	0.206	9.08	—	0.227	4.98	—
0.50	trust	0.207	9.09	0.025	0.231	4.97	0.008
0.50	auth	0.000	0.00	0.013	0.000	0.00	0.021
0.75	none	0.348	10.45	—	0.372	5.58	—
0.75	trust	0.207	2.54	0.892	0.248	0.85	0.950
1.00	none	0.437	11.26	—	0.472	6.01	—
1.00	trust	0.234	2.75	0.942	0.260	0.65	1.000
1.00	auth	0.000	0.00	0.937	0.000	0.00	1.000

13.3 Why detection is a step function

The detector is a **threshold test on rate**. An attacker at greed 0.50 does not exceed `priority_flag_rate` = 0.50, so it is never flagged. Detection jumps from **0.008** → **0.950** between greed 0.50 and 0.75 on the highway. There is no gradual degradation; the attacker simply sits under the step.

13.4 The strongest row in the project

At greed 0.50, **auth** gives EMS deadline-miss 0.000 and 0.00 % drain, while its own behavioural detector reads 0.013–0.021 — i.e. blind.

Read that against the `trust` row directly above it: detection 0.008–0.025, at or **below** the false-positive rate, meaning the detector contributes nothing; the attacker takes **47 % of maximum deadline damage** (0.206 / 0.437 urban) and **81 % of maximum energy drain** (9.08 / 11.26); and **trust makes the outcome marginally worse, not better** (0.206 → 0.207 urban, 0.227 → 0.231 highway).

Authorisation is not detecting the attack. **It does not need to.** That is the difference between a statistical defence and a cryptographic one, in one row.

13.5 Why lowering the threshold cannot fix it

At greed 0.18 the attacker's **per-node behaviour is identical to a real ambulance's**. No rate-based test can separate them at *any* threshold — they emit the same rate by construction. Yet 20 attackers asserting 4 messages each still outnumber 5 ambulances asserting 4 each **four to one**, and 2.4–4.5 % of network energy still drains.

The distinguishing fact is not behavioural at all. It is whether the vehicle holds the role.

PART 14 — AUTHORISATION

14.1 The mechanism

element	what it is
TA	trusted authority; issues certificates, holds the pseudonym→identity mapping for conditional privacy
provisioning credential	$k(V, TA)$, installed out of band when an emergency vehicle is commissioned. A vehicle without it cannot request an EMS certificate
role attribute	EMS — a 1-byte field inside the certificate
pseudonym certificate	122 B explicit form: pseudonym id 8 + compressed pubkey 33 + role 1 + validity 8 + issuer 8 + TA signature 64
assertion	class-2 message + certificate reference + ECDSA signature over (tag, V, pid, role, nonce, target head)
slot grant	head verifies TA signature on the cert, verifies the assertion signature, checks the role, checks freshness, then grants
replay protection	the nonce, plus injective agreement verified in Scyther
rate limit	<code>priority_budget</code> token bucket, so even a compromised <i>authorised</i> ambulance cannot flood (implemented, default 0 = unlimited)

In the simulator this is abstracted as `TrafficModel.is_ems` — a boolean. **Module 3a is where that boolean is given a protocol and proved sound.**

14.2 Why authorisation is categorically different from trust

	trust	authorisation
basis	statistical inference from observed behaviour	verification of an unforgeable credential
can a restrained attacker evade it?	yes — greed 0.50, detection 0.008	no — it never looks at behaviour
false positives?	yes — honest ambulances lose slots	structurally impossible
cost when no attack present	non-zero (0.002 urban)	exactly zero
what it needs to work	enough rounds to accumulate evidence	one signature verification

Trust asks "*has this vehicle been behaving like an ambulance?*" — a question a patient attacker can answer correctly. Authorisation asks "*is this vehicle an ambulance?*" — a question it cannot.

14.3 Why 0.000 at every attacker fraction

Because false demand is **never admitted**. In `_priority_phase`, with `require_ems_auth = True`, admitted demand is `demand_authorised` — the portion from role-holders only. The attacker's assertions are refused **before** the reservation is shared, so:

- genuine EMS demand is never diluted → served in full → delay 2 slots → deadline met;
- the attacker's messages are never forwarded verbatim → no energy externalised → drain 0.00 %.

This is why authorisation's priority-energy column stays flat as attackers rise, while the undefended column climbs 560 → 6342 mJ.

14.4 The p-value

`p = 5.134e-06`, Mann-Whitney U, one-sided (`alternative="less"`), comparing EMS deadline-miss under `auth` versus undefended at 20 % attackers, 12 seeds per arm. **Identical in both scenarios** — and identical before and after crypto was charged.

That exact repetition is not a coincidence and should be explained honestly if asked: with 12 seeds per arm where one arm is **0.000 in every seed** and the other is strictly positive in every seed, the ranks separate completely, so the statistic takes its extreme value and the p-value is the minimum attainable for $n = 12$. It confirms complete separation; it is not evidence of effect *size*. The effect size is the deadline-miss difference itself ($0.437 \rightarrow 0.000$).

PART 15 — URBAN VERSUS HIGHWAY

15.1 The two scenarios

	urban	highway
field	600 × 600 m Manhattan grid, 200 m blocks	1000 × 21 m, 6-lane bidirectional
RSUs	9, grid layout at quadrant centres	5, line layout, every 200 m
tx_range	150 m	100 m
speed	12 ± 3 m/s (4–18)	25 ± 4 m/s (8–33)
mobility	turns at intersections, p_turn = 0.35	lane-holding, wrap-around, closing speeds to 66 m/s
nodes	100	100
FND (CHIRP)	~9.4 rounds	~23.0 rounds
network lifetime	short	~2× longer

15.2 Findings that differ, and why

finding	urban	highway	mechanism
orphan rate (CHIRP)	3.1 %	0.1 %	denser RSU grid vs longer thin road
emergency dl-miss, no bypass	0.874	0.958	highway has almost no accidental orphan fast path
bypass cost	+1.2 / +1.8 %	+0.4 / +0.5 %	d ² vs d ⁴ — see below
crypto + bypass total	+1.63 %	+0.77 %	same
attack drain @ 20 %	11.26 %	6.01 %	same
trust detection	0.942–0.983	1.000	2× lifetime → 2× evidence
trust FPR cost @ 0 attackers	non-zero	0.000	same
authorisation	0.000	0.000	invariant

15.3 The RSU-density finding

The cost of priority is set by RSU density, not by traffic mix.

The highway places 5 RSUs along 1 km, so heads sit almost always **inside** the free-space knee at $d_0 = 87.71$ m and each verbatim forward costs $\epsilon_{fs} \cdot d^2$. The 600 m urban grid, despite having *more* RSUs, spreads them over a 2-D area, so many heads sit **beyond** the knee and pay $\epsilon_{mp} \cdot d^4$ — where each extra bit is disproportionately expensive.

Roughly **3× the relative cost** for the same feature. A deployment that wants cheap priority handling should **buy RSUs, not bandwidth** — and the same mitigation halves the false-priority attack's energy damage.

15.4 What is and is not demonstrated

Demonstrated: - Every finding replicates across two quite different road geometries, RSU densities and speed regimes. - Two findings appear *only* in comparison: the RSU-density cost effect, and trust's false-positive cost being confined to short-lived networks.

NOT demonstrated — state this plainly: - **This is not real-world highway validation.** Both mobility models are synthetic, generated by `src/mobility.py`. There are no replayed SUMO traces, no OpenStreetMap topology, no real vehicle traces. The comparison paper P3 (Jaswani et al.) *does* use SUMO + OSM + NS-2.35; we do not. - **No channel model.** No contention, no collisions, no interference, no fading. Slot exhaustion is modelled honestly because a TDMA schedule genuinely is a finite resource; channel-level flooding is **not modelled at all**. - **No milliseconds, no Mbps, no jitter, no multi-hop route latency.** - Absolute round counts should not be read as realistic OBU lifetimes — the Heinzelman model was designed for short-range sensor motes and charges d^4 at VANET distances.

PART 16 — THE REPRODUCTION GAP

The most methodologically important part of the project. Reproduce with `python3 src/verify_table4.py`.

16.1 The discrepancy

Our reproduction at the fitted 22 frames/round, against Table 4:

Round	LEACH (ours / paper)	LEACH-C (ours / paper)	CSGD-NET (ours / paper)
10	0.35717 / 0.35529	0.35789 / 0.37000	0.35727 / 0.32982
20	0.22262 / 0.22091	0.21573 / 0.28339	0.21602 / 0.26095
30	0.11427 / 0.12717	0.07450 / 0.14785	0.09030 / 0.15064
40	0.03837 / 0.03954	0.00305 / 0.04387	0.01032 / 0.07435
RMSE	0.005	0.049	0.046

LEACH matches. LEACH-C and CSGD-NET do not — ours deplete much faster.

16.2 What we initially believed — and withdrew

`docs/reproducibility.md` originally recorded: *"That gap points at our LEACH-C and CSGD-NET, not at the publication."*
That attribution was tested and is withdrawn. Three independent arguments, all agreeing.

16.3 Argument 1 — the protocol-dependent energy budget is ~0.002 J/node

Per-round network energy under Eqs. (9)–(11) splits in two:

ELECTRONICS	$(2 \cdot (n - n_{ch}) + n_{ch}) \cdot k \cdot E_{elec}$	+ aggregation
AMPLIFIER	$k \cdot \epsilon_{fs} \cdot \Sigma d^2$	

The electronics term does not depend on which nodes are heads. For *any* head set, every member sends one packet, every head receives it and sends one. So the **amplifier is the entire budget a CH-selection algorithm has to play with.**

Measured by re-running with the amplifier effectively disabled ($\epsilon_{fs} \rightarrow 1e-30$, d_0 pushed far beyond the field):

protocol	R10 actual	R10 zero-amplifier	budget
LEACH	0.35723	0.35927	0.00204 J/node
LEACH-C	0.35790	0.35920	0.00130 J/node
CSGD-NET	0.35723	0.35920	0.00197 J/node

16.4 Argument 2 — Table 4's spread is 20–44× that budget

spread at R10	value	vs budget
ours, three protocols	0.00068	within
Table 4, LEACH/LEACH-C/CSGD-NET	0.04018	19.7×
Table 4, all six columns	0.09018	44.2×

The decisive case needs no estimate. Set the amplifier to zero — perfect, physically unreachable clustering — and LEACH-C reaches **0.35920** at round 10 against Table 4's **0.37000**. Still short by **0.0108 J/node**. No cluster-head selection algorithm can close that, because the energy it would have to save is not spent on anything clustering controls.

16.5 Argument 3 — two columns violate monotonicity

Drain per round = (alive nodes) × (per-node cost). Nodes only die, and per-node cost is dominated by the constant electronics term. **Drain cannot rise between intervals.**

protocol	source	drops R1–10, 10–20, 20–30, 30–40	max rise
LEACH	paper	0.1447 0.1344 0.0937 0.0876	−6.5 %
LEACH	ours	0.1428 0.1342 0.1066 0.0757	−6.0 %
LEACH-C	paper	0.1300 0.0866 0.1355 0.1040	+56.5 %
LEACH-C	ours	0.1421 0.1421 0.1412 0.0717	0.0 %
CSGD-NET	paper	0.1702 0.0689 0.1103 0.0763	+60.2 %
CSGD-NET	ours	0.1428 0.1414 0.1258 0.0797	−0.9 %
HEED / PSO / GA	paper	exactly constant	0.0 %

The correlation is the diagnosis. The only Table 4 column internally consistent with the paper's own model is LEACH — and LEACH is the only column we reproduce. The two we fail on are exactly the two that violate a constraint the model cannot violate.

16.6 Hypotheses tested and rejected

Alternative readings of "Remaining energy" — the paper never defines it. Six readings tested (mean over all nodes / mean over survivors / network total, each × two round alignments, since row 1 = 0.5 hints the row may be *before* the round), refitting frames/round for each:

metric	align	frames	LEACH	LEACH-C	CSGD-NET
all	0 (<i>current</i>)	22	0.0083	0.0490	0.0466
all	1	23	0.0083	0.0489	0.0476
alive	0	24	0.0126	0.0610	0.0460
alive	1	25	0.0085	0.0588	0.0448
total	0	45	22.93	22.78	22.93
total	1	45	10.90	10.51	10.84

Best LEACH-C across all six: 0.0489 against the current reading's 0.0490. **Nothing closes.** "Network total" is excluded outright — it would put the column near 50 J. Choosing different metrics for different protocols would be incoherent.

A defect in our LEACH-C — rejected. It produces the **shortest member links of any protocol tested** (6.09 m mean vs LEACH's 12.88 m) and the **best first-node-death** (28.3 rounds vs LEACH's 11.4). It is clustering well.

16.7 The ~300 m field — a hypothesis, clearly labelled

Sweeping field size (refitting frames/round at each, so a larger field is not penalised merely for draining faster):

area	frames	LEACH RMSE	LEACH-C RMSE	CSGD-NET RMSE	spread @R10
50 m	22	0.0054	0.0486	0.0461	0.00068
100 m	20	0.0077	0.0349	0.0383	0.00283
200 m	16	0.0120	0.0140	0.0327	0.00943
300 m	11	0.0105	0.0209	0.0221	0.05556

A ~300 m square **does** reproduce the spread (Table 4's own spread is 0.04018), because past d0 = 87.7 m the d4 branch makes clustering quality dominate cost. At 300 m, CSGD-NET's R10 lands at 0.32951 against the paper's 0.32982.

BUT: Table 3 explicitly states "Simulation Zone 50 × 50 m", with nothing contradicting it anywhere in the paper. Adopting 300 m would be fitting a curve against an explicitly published parameter. **We did not make that change.**

16.8 Epistemic status of each statement

statement	status
Our LEACH reproduces Table 4's LEACH column to RMSE 0.005	observed fact
The electronics term is protocol-independent	mathematical fact , from Eqs. (9)–(11)
The amplifier budget at R10 is 0.00130–0.00204 J/node	measured , by zero-amplifier re-run
Table 4's spread exceeds that budget by 20–44×	arithmetic on measured + published values
Zero-amplifier LEACH-C still falls 0.0108 J/node short	measured
Paper's LEACH-C/CSGD-NET drain rises +56 %/ +60 % between intervals	arithmetic on published values
Drain cannot rise under this model	mathematical fact
Therefore Table 4's LEACH-C/CSGD-NET columns are not producible by the paper's own equations at its own parameters	conclusion, follows from the above
A ~300 m field would produce them	hypothesis , supported by a sweep
Table 4 was produced at a different field size than Table 3 documents	speculation — plausible, unverifiable from outside, not claimed
The values were fabricated	NOT CLAIMED, NOT INVESTIGATED, and must not be alleged

16.9 Why we deliberately changed no simulator code

The task was diagnosis. The diagnosis is that **no correction is warranted** — the target is unreachable, so "fixing" our implementation to hit it would be curve-fitting against an unachievable number. Acceptance criterion used: *a column is reproduced if $RMSE \leq 0.01$* (the level achieved on the one consistent column) **and the target is achievable under the paper's own model**. LEACH passes both; LEACH-C and CSGD-NET fail the second precondition, so the criterion is **inapplicable rather than failed**.

16.10 Consequence, and a side finding

Table 4 cannot serve as a baseline. Every comparison in this project uses our own re-implementations — which it already did.

Side finding worth stating at a viva: mean residual energy over all nodes is a **perverse metric**. A dead node stops consuming, so a protocol that kills nodes early has fewer consumers and *appears* to conserve energy. Our LEACH-C keeps 94.8 nodes alive at round 30 against LEACH's 79.6 — better balance, therefore more consumers, therefore a faster fall in the mean. **It trails LEACH because it is doing its job**. This is a further reason Module 1 reports first/half/last-node-death and energy per delivered reading instead.

PART 17 — FORMAL VERIFICATION (MODULE 3a)

Scyther v1.3.0, `--max-runs=4` . 32 claims verified, 8 failing by design.

```
SCYTHER=/path/to/Scyther/scyther-mac python3 formal/run_scyther.py
```

`formal/run_scyther.py` compares results against the expected set recorded here and **exits non-zero if anything moves**, so this section cannot go stale.

17.1 The adversary model and the claim vocabulary

Dolev-Yao. The adversary is the network: it sees every message, can intercept, reorder, replay, and construct any term derivable from what it knows. It cannot break cryptography. Honest agents in the claim run are not compromised under Scyther's default.

Lowe's authentication hierarchy, as Scyther implements it:

claim	meaning
Alive	the intended peer ran the protocol at some point
Weakagree	the peer ran the protocol <i>with the claimer</i>
Niagree	non-injective agreement — peers agree on the data values
Nisynch	non-injective synchronisation — every preceding send/rcv in every role matched , in order

We report all four on every role so a failure can be **located** on the hierarchy rather than merely observed.

17.2 The models

`register.spdl` — certificate issuance — 8/8 verified

Roles **V** (vehicle), **TA**.

1. $V \rightarrow TA : V, \{TagCertReq, V, pid, EMS, nv\}k(V, TA)$
2. $TA \rightarrow V : \{TagCert, pid, EMS, V, nv\}sk(TA)$

Claims: Alive, Weakagree, Niagree, Nisynch on both roles — **all Ok**.

Why message 1 is authenticated under $k(V, TA)$: without it the model would describe a TA that issues an EMS certificate to anyone who asks, which would make Module 5's entire authorisation defence **vacuous** — the false-priority attacker would simply register as an ambulance. The provisioning credential is installed out of band when an emergency vehicle is commissioned.

`priority.spdl` — the slot grant — the load-bearing model

Roles **V**, **TA**, **C** (cluster head).

1. $V \rightarrow TA : V, \{TagCertReq, V, pid, EMS\}k(V, TA)$
2. $TA \rightarrow V : \{TagCert, pid, EMS, V\}sk(TA)$
3. $V \rightarrow C : V, \{TagCert, pid, EMS, V\}sk(TA), nv, \{TagAssert, V, pid, EMS, nv, C\}sk(V)$
4. $C \rightarrow V : \{TagGrant, nv, nc, V, C\}sk(C)$

role	Alive	Weakagree	Niagree	Nisynch
V	Ok	Fail	Fail	Fail
TA	Fail	Fail	—	—
C	Ok	Fail	Ok	Ok

`claim_C4(C, Nisynch)` — Ok, proof of correctness. This is the whole point.

What C4 establishes, precisely

Over a three-role protocol, Nisynch requires every preceding event in **every** role to have a matching partner, in order. So if C4 holds, then **whenever a cluster head completes and grants a reserved slot**:

- there was a **real TA run** that issued that role attribute to that vehicle;
- there was a **real vehicle run** that asserted it;
- all three agree on pseudonym, role and nonce;
- and the nonce rules out replay.

In one sentence: **a head grants a reserved emergency slot only to a vehicle holding a TA-issued EMS role attribute, and never to a replay of one.** That is exactly the assumption `require_ems_auth` encodes as a boolean in the simulator.

Note the assertion signature binds the **target head C**, so an assertion overheard in one cluster cannot be relayed into another.

`priority-assert.spdl` — the exchange in isolation — 8/8 verified

Two roles, $V \rightarrow C$, certificate absent. Exists to prove the six failures above are structural rather than vulnerabilities (Part 20).

`aggregate.spdl` — the three data-path message types — 12/12 verified

```
chirp-member      M → C : M, reading, nm, {TagReading, M, reading, nm, C}k(M,C)
chirp-aggregate C → R : C, agg, nc, {TagAggregate, C, agg, nc, R}k(C,R)
chirp-verbatim V → R : V, pid, msg, nv,
                        {TagEmergency, V, pid, EMS, msg, nv, R}sk(V)
```

All four claims Ok on all three. `chirp-verbatim R.Nisynch` is the interesting one: the RSU authenticates the emergency vehicle **end to end, through a relay it does not trust at all**.

The relay is deliberately **not** a Scyther role — see Part 20.

`fusion-limit.spdl` — deliberate negative result — 2/2 failed, as designed

See Part 19.

17.3 What Scyther cannot do here, and is therefore not claimed

- **The TA's out-of-band vetting.** That an EMS credential reached only a real ambulance is a property of the registration desk, not of any message exchange. What is verified is that the role attribute cannot be **forged, replayed, or transplanted** onto another vehicle's assertion.
 - **Conditional privacy and unlinkability.** These are *equivalence* properties. Scyther proves secrecy and authentication only; unlinkability needs ProVerif's observational equivalence or Tamarin's diff mode. **Not claimed.**
 - **Pseudonym confidentiality.** A signature is not an envelope — `{m}sk(TA)` is readable by anyone holding `pk(TA)`, which is everyone. The pseudonym is public by design. No `Secret` claim is made on it; one would fail, correctly.
 - **ECC itself.** Symbolic verification assumes perfect cryptography. Point multiplication is an abstract one-way function. This verifies the *message flow*, not the curve.
 - **Bounded search.** `--max-runs=4`. Every result is "no attack within four concurrent runs" — the standard bound, but a bound.
-

PART 18 — THE REAL BUG SCYTHER FOUND

This is the concrete return on doing the formal work, and it should be stated as a finding rather than buried as a fix.

18.1 The original design

The first version of `aggregate.spdl` MAC'd both data-path messages with the same structure:

```
member reading    : {M, reading, nm, C}k(M,C)
fused aggregate   : {C, agg,      nc, R}k(C,R)
```

Both are `{sender, value, nonce, receiver}k(sender, receiver)`. **Structurally identical — no domain separation.**

18.2 The attack trace

Scyther produced, verbatim from the `--dot-output` graph:

```
Run #2: Alice in role C of chirp-AGGREGATE
        send_1 to Charlie
        (Alice, agg#2, nc#2, {Alice, agg#2, nc#2, Charlie}k(Alice,Charlie))

Run #1: Charlie in role C of chirp-MEMBER
        recv_1 from Alice
        (Alice, agg#2, nc#2, {Alice, agg#2, nc#2, Charlie}k(Alice,Charlie))

redirect node: "select {Alice,agg#2,nc#2,Charlie}k(Alice,Charlie)"
```

A message Alice emitted as a cluster head forwarding a fused aggregate to RSU Charlie was accepted by Charlie as a member reading from Alice. And symmetrically in the other direction.

Failure signature: `Niagree` and `Nisynch` failed on both legs while `Alive` and `Weakagree` passed — the receiver had the right peer but the **wrong message meaning**.

18.3 Why the simulator could never see it

In `src/network.py` a member reading and a fused aggregate are **different Python variables reached by different code paths**. There is no bitstring, no wire format, and therefore no possibility of confusing one for the other. The simulator cannot represent the bug, so no amount of simulation would find it.

This is the argument for formal verification in this project, made concrete: it is not that Scyther proved something we already believed — it is that Scyther found something we could not have found any other way.

18.4 The fix and the re-verification

Distinct constant tags inside every MAC and signature, so the message types occupy disjoint formats:

```
usertype MsgTag;
const TagReading, TagAggregate, TagEmergency: MsgTag;      // aggregate.spdl
const TagCertReq, TagCert, TagAssert, TagGrant: MsgTag;    // priority.spdl, register.spdl
```

After the fix: **all 12 claims in `aggregate.spdl` verify**, and the whole suite verifies **identically whether the files are checked separately or concatenated into one file** — confirming no cross-protocol attacks remain anywhere in the suite.

PART 19 — THE FUSION LIMITATION

`formal/fusion-limit.spdl`. This model is expected to fail. The failure is the result.

19.1 The question

Can the RSU authenticate the **member** whose reading is inside a fused packet?

```
M --{TagReading, M, reading, nm, C}k(M,C)--> C
C --{TagAggregate, C, agg, nc, R}k(C,R)--> R
```

Both legs are individually sound — `aggregate.spdl` verifies them. But the fused value `agg` is **fresh at the head**, not a recoverable function of `reading`, because fusion is lossy and non-invertible.

19.2 The result

`claim_R3(R, Niagree)` and `claim_R4(R, Nisynch)` — **Fail, at least 3 attacks.**

19.3 What it means

Aggregate authentication $\neq$ origin authentication.

- The aggregate MAC attests to **who sent the value**.
- It attests **nothing** about whether the value faithfully summarises the readings that produced it.
- Therefore **a compromised cluster head can fabricate a fused value and no aggregate MAC will detect it.**

This is a property of *aggregation itself*, not a flaw in the MAC, and it is not fixable by choosing a better MAC. It is why a "no attacks" result on `chirp-aggregate` must **not** be read as "the fused value is trustworthy".

19.4 How this formally justifies the verbatim emergency path

`chirp-verbatim` `R.Nisynch` **verifies**. Because a class-2 message is never transformed, the vehicle's own signature survives the relay to the RSU.

So the class-2 bypass gains a **second, independent justification that has nothing to do with latency**:

The bypass is the only path in CHIRP that gives the RSU end-to-end authentication of the originating vehicle. **Fusion buys energy and costs provenance.**

That sentence is worth memorising for a viva — it converts what looks like a performance hack into a security-architectural necessity.

PART 20 — THE TA / WEAKAGREE MODELLING ISSUE

Do not present this as a protocol vulnerability. It is a modelling/partnering limitation, and it was diagnosed by experiment, not assumption.

20.1 What failed

In `priority.spdl`: C's Weakagree, and V's and TA's agreement claims — six claims — while C's Niagree and Nisynch passed.

That inverts Lowe's hierarchy (Weakagree is normally weaker than Niagree), which is what made it worth investigating rather than explaining away.

20.2 Two wrong diagnoses, both tested and discarded

1. "It's a partnering artifact with no adversary action." Tested by counting adversary `select / redirect` nodes in each attack graph. C's Weakagree trace **did** contain adversary action (`fake sender Alice, redirect to Charlie`). Hypothesis rejected.
2. "The assertion signature doesn't bind V's own identity." The signature was over (`TagAssert, pid, EMS, nv, C`) — the sender appeared only in the unsigned outer tuple. Tested by rebuilding it as (`TagAssert, V, pid, EMS, nv, C`). **The claims still failed.** Hypothesis rejected. (The binding was kept anyway as standard hardening, after verifying it changes no result.)

20.3 The actual cause, confirmed by experiment

The TA never learns which cluster head the vehicle will later talk to.

Registration happens offline and once, long before any particular head is in the vehicle's neighbourhood. Message 1 carries no head identity and nothing ever tells the TA one. Weakagree over three roles asks whether **every** other role is "running the protocol *with* the claimer" — and the TA is not running with any head, because it has never heard of one.

Confirmation: delete the TA role, keep the assertion exchange otherwise identical, and **all six claims verify**. That experiment is preserved as `formal/priority-assert.spdl` (8/8 Ok).

20.4 Why Niagree/Nisynch are the right load-bearing claims

Niagree and Nisynch concern **data agreement and message ordering**, both of which *do* match across all three roles. They hold **with the TA in the model**, i.e. with certificate provenance included — which is precisely the property we need.

Weakagree's partnering relation is the wrong question to ask of a two-phase protocol whose phases are separated in time and have disjoint participants.

20.5 The three models together

model	covers	result
<code>register.spdl</code>	certificate issuance	8/8 Ok
<code>priority.spdl</code>	head's agreement with provenance	C: Niagree, Nisynch Ok
<code>priority-assert.spdl</code>	the assertion exchange alone	8/8 Ok

Nothing papered over: issuance is sound, the exchange is sound, and the head's guarantee holds transitively back to a real TA issuance.

Why the certificate is absent from `priority-assert.spdl`: a Scyther role can only send terms it can *derive*, and V cannot construct `{...}sk(TA)` — it does not hold the TA's signing key, which is the entire point of a certificate. Modelling it without the TA role would mean declaring it a global constant and thereby handing it to the adversary too.

PART 21 — MODULE 3b: CRYPTO COST METHODOLOGY

```
python3 src/crypto_bench.py --seconds 3
```

Uses only the `openssl` CLI and the Python standard library — **no new dependency**. Output: `results/crypto_bench.json`.

21.1 MEASURED — sizes, from real keys and real signatures

OpenSSL 3.6.1 (27 Jan 2026).

item	bytes	status
ECDSA P-256 signature, raw <code>r s</code> (IEEE 1609.2 form)	64	measured
ECDSA P-256 signature, DER	71 (+7 framing)	measured
public key, compressed point (DER)	59 (raw point 33)	measured
public key, uncompressed (DER)	91 (raw point 65)	measured
HMAC-SHA256 tag, full	32	definitional
aggregate MAC, truncated to 128 bits	16	design choice

Pseudonym certificate, explicit form — 122 B:

field	bytes
pseudonym id	8
public key (compressed point)	33
role attribute	1
validity start/end	8
issuer id	8
TA signature (ECDSA P-256)	64
total	122

Derived wire overhead:

```
sig_bits = (64 + 8) × 8 = 576      # signature + 8 B HashedId8 cert digest
mac_bits = 16      × 8 = 128
```

IEEE 1609.2 allows either the full certificate or an 8-byte HashedId8 digest; a deployment sends the full certificate periodically and the digest otherwise. The digest variant is the default. **Pessimistic variant** (full cert on every class-2 message): `sig_bits = 1488` — measured and reported, not used.

21.2 MEASURED — timings, on this machine

operation	ops/s	μs
ECDSA P-256 sign	29,901	33.4
ECDSA P-256 verify	10,269	97.4
HMAC-SHA256 over an 800 B packet	604,964	1.65

Verification is 2.9× the cost of signing, and verification is the direction CHIRP does most — a head verifies every class-2 assertion it admits, and the RSU verifies every verbatim message. A design that optimised signing would be optimising the wrong direction.

Caveat, stated in the tool's own output: this OpenSSL binary is x86-64 running under Rosetta on an arm64 host, so it **understates** a native build.

21.3 ESTIMATED — computation energy. NOT MEASURED.

This section is explicitly an extrapolation. Do not present it as a measurement.

Energy per operation = time × CPU power. **CPU power is an assumption**, so it is swept. **OBU slowdown relative to this machine is also an assumption** — embedded ARM cores run ECDSA P-256 roughly 20–50× slower than a desktop core, a figure taken from the literature, not measured here.

Reference: transmitting the 72 B class-2 overhead at 90 m costs **0.078 mJ** (computed from the project's own radio model — that part is derived, not assumed).

OBU slowdown	verify	0.25 W	0.5 W	1.0 W	2.0 W
1× (this machine)	97 μs	0.024	0.049	0.097	0.195 mJ
10×	974 μs	0.243	0.487	0.974	1.948 mJ
25×	2435 μs	0.609	1.217	2.435	4.869 mJ
50×	4869 μs	1.217	2.435	4.869	9.738 mJ

21.4 The break-even — the assumption-light way to state it

Verification costs more energy than transmitting its own signature once verify time exceeds:

CPU power	break-even verify time	
0.25 W	312 μs	
0.5 W	156 μs	
1.0 W	78 μs	already exceeded on this laptop
2.0 W	39 μs	already exceeded on this laptop

At a plausible **25× slowdown and 0.5 W**, verification costs **1.22 mJ against 0.078 mJ** to transmit what it verifies — about 15×.

The conclusion that does not depend on the assumptions: the base paper states (§3.3) that "*computations and processing do not incur energy expenditure*". For a protocol with no cryptography that is defensible. **With a crypto layer, the cost is dominated by computation, and a radio-only model is structurally incapable of showing it.**

21.5 What is NOT done

Computation energy is measured/estimated and reported; it is NOT charged in the simulator. Charging it would touch every protocol and every result — all nodes sign and verify, not just emergency vehicles — and would move the Module 1 and Module 2 baselines the whole comparison rests on. That is a deliberate scope boundary, with the expected magnitude documented so the decision can be made on evidence: at 25× and 0.5 W, one verification per class-2 message is ~1.2 mJ against a cluster head's ~88 mJ per round.

No hardware acceleration is modelled. Automotive HSMs with ECDSA engines would move the break-even substantially.

PART 22 — CRYPTO'S RADIO COST

22.1 Method

Four-step decomposition, each row adding one thing to the row above, **16 seeds**, CHIRP, 5 % EMS, paired **Wilcoxon signed-rank** test across seeds (paired because each seed gives the same mobility trace in both arms).

22.2 Results

Urban — mJ per delivered reading

configuration	mJ/reading	delta	cumulative	p
no crypto, no bypass	1.3168	—	—	—
+ aggregate MAC	1.3146	−0.17 %	−0.17 %	0.63
+ bypass, no signature	1.3288	+1.08 %	+0.91 %	0.083
+ per-message signature	1.3383	+0.71 %	+1.63 %	0.058

Highway

configuration	mJ/reading	delta	cumulative	p
no crypto, no bypass	0.7269	—	—	—
+ aggregate MAC	0.7288	+0.26 %	+0.26 %	0.004
+ bypass, no signature	0.7326	+0.52 %	+0.77 %	0.002
+ per-message signature	0.7325	−0.00 %	+0.77 %	0.94

22.3 Reading them

Everything together costs under 1.7 % of network energy. The whole crypto layer plus the priority bypass is a rounding error against the radio budget.

The per-message signature is statistically undetectable on the highway ($p = 0.94$) and **only marginal in urban** ($p = 0.058$, i.e. does not reach the conventional 0.05 threshold).

Language discipline: do **not** say the signature is "free". Say: "*we could not detect a per-message signature cost at 16 seeds ($p = 0.94$ highway, 0.058 urban).*" Absence of a detected effect is not proof of no effect — a larger seed count might resolve the urban +0.71 %.

Why so small: 576 extra bits ride on a transmission already carrying a 6400-bit payload **plus** a fixed `k·Eelec` electronics term that dominates at short range, and class-2 traffic is only **0.84 %** of all readings at the defaults (5 % EMS × 4 of 22 messages).

The aggregate MAC sits at the edge of measurability, and which side depends on the scenario. Highway: +0.26 % at $p = 0.004$. Urban: −0.17 % at $p = 0.63$ — noise. Urban networks die at round ~9.6 against the highway's ~23, so there is far less averaging and variance swamps a sub-percent effect. **Reporting the urban figure as a *reduction* would be reading noise as signal.**

PART 23 — CRYPTO × ATTACK INTERACTION

23.1 The unanticipated result

Charging the measured `sig_bits` made the false-priority attack **more expensive for its victims**:

scenario	net drain @ 20 % attackers, <code>sig_bits = 0</code>	with <code>sig_bits = 576</code>
urban	10.41 %	11.26 %
highway	5.52 %	6.01 %

(Both figures are final; the second column is what is in `results/priority_defence_*.json` today.)

23.2 Why

Every falsely-urgent message now carries a signature, and the honest head must forward it **verbatim** — signature included — because that is what the bypass does. So the added bytes are paid by:

- the **head**, on every fake-urgent forward, and
- **not** by the attacker, which pays only its ordinary member transmit.

The attacker's marginal cost of attaching a signature is negligible; the victim's marginal cost of relaying it is not. **Crypto amplifies the externalisation.**

23.3 Why authorisation removes it entirely

Under `require_ems_auth`, the false claim is **refused before the forward ever happens**. Admitted demand is computed from `demand_authorised` only, so unauthorised class-2 demand never enters the reservation, never gets bypassed, and never gets forwarded verbatim. Hence `drain % = 0.00` and priority energy flat at ~530–573 mJ (urban) across every attacker fraction.

The security lesson generalises: adding integrity protection to a channel an attacker can induce you to use makes the attack *worse*, not better, unless you also add an **admission** decision. Authentication of a message is not the same as authorisation to send it.

PART 24 — ALL MAJOR CORRECTIONS

Claims that were made and later overturned by measurement. **Knowing these prevents repeating them at a viva.**

#	Original belief	What the experiment showed	Final interpretation
1	The LEACH-C/CSGD-NET reproduction gap is a defect in our implementation	Amplifier budget 0.00130–0.00204 J/node; Table 4 spread 20–44× that; zero-amplifier LEACH-C still 0.0108 short; paper's drain rises +56 %/+60 %	The gap is in the published table, not our code. Only LEACH is a usable target
2	The base paper's arithmetic is sound	LEACH column reproduces at RMSE 0.005; the other two are unreachable	Scoped: <i>the depletion rate is achievable and the LEACH column is sound</i> ; the LEACH-C/CSGD-NET columns' arithmetic is not
3	Stability-triggered re-clustering saves energy	Raising the ceiling 1→5 cost 26 % urban FND, 35 % highway FND, to save 0.28 mJ; head role costs 311 × a re-election	Reverted. <code>recluster_max_rounds = 1</code> , plus a cooldown. Code kept, negative result documented
4	LET-aware member association improves stability	Monotonically worse on every metric (FND 16.1→14.1, mJ/reading 0.728→0.792)	Reverted. <code>assoc_let_weight = 0</code>
5	A survival-prediction fitness term protects the weakest head	Monotonically worse; the optimiser raised survival by orphaning members (3.9 %→6.0 %)	Reverted. <code>w_survival = 0</code>
6	Trust detection of false-priority attackers will reduce the damage	Detection 0.925, deadline-miss 0.452 → 0.463 — unchanged	Detection ≠ mitigation. Needed an enforcement path on the <i>slot grant</i> , not CH eligibility
7	CHIRP pays less than half what CSGD-NET pays for the bypass (urban), thanks to its energy-scored RSU term	With crypto charged the ordering reverses : CHIRP +1.8 % vs CSGD-NET +1.2 %	Withdrawn. The fixed MAC cost lands against CHIRP's lower baseline, and CHIRP orphans almost nothing so more traffic needs bypassing
8	Trust gating costs 0.015 EMS deadline-miss at zero attackers	Re-run gave 0.002 ; crypto cannot affect a trust FPR	Seed noise at 12 seeds. Claim is now directional only , carries no number
9	Filling in <code>sig_bits</code> will raise the cost of the bypass	Per-message signature statistically undetectable (p = 0.94 highway)	The prediction was wrong. Transmission cost is negligible; computation is where the cost is
10	Scyther is a 2014 tool, x86-only, needs Rosetta on Apple Silicon	v1.3.0 released Feb 2026 with a native arm64 macOS binary	Ran directly. This error was told to the user <i>before</i> they chose an option, so it affected a decision
11	<code>chirp-verbatim</code> 's failures were a flaw in the protocol	Modelling the relay as a role makes Nisynch require the relay to be honest; two-role V→R verifies 4/4	Modelling error, not protocol flaw. Under Dolev-Yao the network already <i>is</i> the adversary
12	C's Weakagree failure is a partnering artifact with no adversary action	The trace did contain a redirect node	Hypothesis rejected — but so was the next one
13	C's Weakagree fails because the assertion doesn't bind V's identity	Binding V changed nothing	Hypothesis rejected. Actual cause : the TA never learns the head's identity (Part 20), confirmed by deleting the TA role
14	Urban findings would generalise without checking	Two findings appear only on comparison: RSU-density cost effect; trust FPR cost confined to short-lived networks	Both scenarios are needed; neither alone is sufficient

Historical numbers that must NOT be quoted as current: urban bypass +2.1 %/+0.9 % (pre-crypto); highway +0.4 %/+0.6 % (pre-crypto); urban attack drain 10.41 % (pre-crypto); trust FPR cost 0.015; Module 1 urban table values from `results/module1_urban.json` (3-seed smoke test).

PART 25 — FINAL NUMBERS DATABASE

Every number below was read directly out of the named file. Where a document and a data file disagree, **both are shown and the disagreement is explained**.

25.1 Baseline / Table 4 reproduction — `results/baseline.json` (30 seeds)

Mean residual energy per node:

protocol	R1	R10	R20	R30	R40	R50	FND	HND	LND
LEACH	0.48573	0.35717	0.22262	0.11427	0.03837	0.00405	11.4	40.1	58.9
LEACH-C	0.48580	0.35789	0.21573	0.07450	0.00305	0.00052	28.3	36.2	54.4
HEED	0.48561	0.35707	0.21432	0.07774	0.00051	0.00001	21.3	37.0	43.5
PSO	0.48579	0.35785	0.21585	0.08262	0.00826	0.00037	20.5	36.6	53.4
GA	0.48578	0.35774	0.21620	0.09014	0.01621	0.00130	18.3	36.7	56.4
CSGD-NET	0.48572	0.35727	0.21602	0.09030	0.01032	0.00006	16.3	37.5	49.4

RMSE vs Table 4 (from `src/verify_table4.py`, 10 seeds): LEACH **0.005**, LEACH-C **0.049**, CSGD-NET **0.046**.

25.2 Module 1 — CH election

Highway, 20 seeds — `results/module1_highway.json` — matches `docs/module1.md`. Table in Part 5.

Urban — **DISAGREEMENT**, investigated:

source	LEACH FND	CHIRP FND	CHIRP mJ/reading	CHIRP orphan %
<code>docs/module1.md</code> (20 seeds)	8.9	9.4	1.2938	3.8
<code>results/module1_urban.json</code>	8.7	9.3	1.2866	3.1

Resolution: the JSON is a **3-seed smoke test** run to confirm the runner still executed during the reproduction-gap task, which was then committed in `9ee0d09`. `docs/module1.md` (20 seeds) is **authoritative**. Action item: regenerate. Highway is unaffected.

Ablation, highway, 10 seeds — `results/module1_ablation_highway.json` — matches `docs/module1.md`. Table in Part 5.

25.3 Module 2 — trust

DISAGREEMENT, same cause.

source	20 % CHIRP+trust PDR	detect	FPR
<code>docs/module2.md</code> (12 seeds)	0.957	0.815	0.033
<code>results/module2_urban.json</code>	0.945	0.836	0.029

Resolution: JSON is a **3-seed smoke test**. `docs/module2.md` is authoritative. Direction and significance are unaffected (both show CHIRP+trust above CSGD-NET).

Attacker energy advantage (`e_adv`, from the JSON, 3 seeds — directionally consistent with the 12-seed doc table): 1.13→2.26 at 10 %, 1.30→2.53 at 20 %, 1.42→2.96 at 30 %, 1.31→2.11 at 40 % (CSGD-NET → CHIRP+trust).

25.4 Delay / coverage inversion — `docs/priority.md` Finding 1 (6 seeds)

Table in Part 8.6. Not stored as JSON — produced by an inline script. **Regenerating it as a committed artifact is an open action item.**

25.5 Priority bypass — `results/priority_bypass_{urban,highway}.json` (12 seeds)

Table in Part 10.3. Matches `docs/priority.md` exactly.

25.6 Attack / defence — `results/priority_defence_{urban,highway}.json` (12 seeds)

Full tables in Part 11.4. Match `docs/priority.md` exactly.

25.7 Greed sweep — `results/priority_greed_{urban,highway}.json` (12 seeds)

Table in Part 13.2. Matches `docs/priority.md` exactly.

25.8 Crypto — `results/crypto_bench.json`

quantity	value
openssl	OpenSSL 3.6.1 (27 Jan 2026)
sig raw / DER	64 / 71 B
pubkey compressed / uncompressed DER	59 / 91 B
certificate total	122 B
<code>sig_bits_digest</code>	576
<code>sig_bits_full_cert</code>	1488
<code>mac_bits</code>	128
ECDSA sign	29,901 /s — 33.44 μ s
ECDSA verify	10,269 /s — 97.38 μs
HMAC-SHA256 / 800 B	604,964 /s — 1.65 μ s
radio mJ for 576 b @ 30/60/90/120/150 m	0.0340 / 0.0495 / 0.0779 / 0.1841 / 0.4079

Decomposition tables (16 seeds, Wilcoxon) in Part 22 — **produced by an inline script, not stored as JSON. Open action item.**

25.9 Formal verification — produced by `formal/run_scyther.py`

model	verified	failed (by design)
<code>register.spdl</code>	8	0
<code>priority.spdl</code>	4	6
<code>priority-assert.spdl</code>	8	0
<code>aggregate.spdl</code>	12	0
<code>fusion-limit.spdl</code>	0	2
total	32	8

25.10 Configuration — `src/config.py` defaults

group	key values
network	<code>n_nodes 100</code> , <code>area 50×50</code> (static), <code>initial_energy 0.5 J</code> , <code>packet_bits 6400</code>
radio	<code>e_elec 5e-08</code> , <code>eps_fs 1e-11</code> , <code>eps_mp 1.3e-15</code> , <code>e_da 5e-09</code> , <code>d0 87.71 m</code>
TDMA	<code>packets_per_round 22</code> , <code>ctrl_bits 200</code>
clustering	<code>ch_percent 0.1</code> → <code>n_ch 10</code> , <code>fitness_mode "C"</code>
cuckoo	<code>n_cuckoos 10</code> , <code>max_iterations 20</code> , <code>pa 0.25</code> , <code>levy_beta 1.5</code> , <code>levy_alpha 0.01</code> , <code>gauss_sigma 0.05</code>
CHIRP fitness	<code>w_energy .30</code> , <code>w_rsu .10</code> , <code>w_intra .35</code> , <code>w_let .05</code> , <code>w_balance .10</code> , <code>w_trust .10</code> , <code>w_survival 0</code>
CHIRP control	<code>ch_cooldown 3</code> , <code>recluster_max_rounds 1</code> , <code>assoc_let_weight 0</code> , <code>energy_stat "mean"</code>
trust	<code>watchdog_p .85</code> , <code>watchdog_err .05</code> , <code>obs_per_round 22</code> , <code>trust_decay .90</code> , <code>decay_unobserved False</code> , <code>trust_penalty 3.0</code> , <code>trust_gate 0.40</code> , <code>cred_sharpness 4.0</code> , <code>trust_warmup 8</code> , <code>eval_alive_frac 0.5</code>
adversary	<code>attacker_frac 0</code> , <code>greyhole_drop .5</code> , <code>onoff_period 8</code> , <code>falsepriority_rate 1.0</code>
priority	<code>ems_frac 0</code> , <code>ems_msgs_per_round 4</code> , <code>emergency_slots 1</code> , <code>deadline_slots 4.0</code>
crypto	<code>sig_bits 576</code> , <code>mac_bits 128</code>
authorisation	<code>require_ems_auth False</code> , <code>priority_trust_gate False</code> , <code>priority_budget 0</code> , <code>priority_window 5</code> , <code>priority_flag_rate 0.50</code> , <code>priority_penalty 2.0</code>

Scenario overrides live in `SCENARIOS` in `src/run_module1.py`:

```
"highway": area 1000×21, n_lanes 6, n_rsus 5, tx_range 100,
            speed 25±4 (8–33), E0 0.5, n_nodes 100
"urban":   area 600×600, block 200, p_turn 0.35, n_rsus 9, tx_range 150,
            speed 12±3 (4–18), E0 0.5, n_nodes 100
```

PART 26 — EXPERIMENT COMMAND DATABASE

```
pip3 install numpy scipy matplotlib scikit-learn
```

(`scikit-learn` is listed but not currently imported anywhere — it was provisioned for the unbuilt Module 4.)

#	command	what it runs	seeds	output
1	<code>python3 src/run_baseline.py --seeds 30</code>	Table 4 reproduction, 6 protocols, static 50×50 m	30	<code>results/baseline.json</code> + console table with Table 4 in brackets
2	<code>python3 src/run_module1.py --scenario highway --seeds 20</code>	CH election, 7 protocols, highway	20	<code>results/module1_highway.json</code>
3	<code>python3 src/run_module1.py --scenario urban --seeds 20 --rounds 150</code>	same, urban	20	<code>results/module1_urban.json</code>
4	<code>python3 src/run_module1.py --scenario highway --seeds 10 --ablate</code>	drop-one-term weight ablation	10	<code>results/module1_ablation_highway.json</code>
5	<code>python3 src/run_module1.py --scenario highway --seeds 8 --sweep-assoc</code>	LET-association sweep (negative result)	8	console
6	<code>python3 src/diagnose_fnd.py</code>	identity/history of the first node to die	—	console
7	<code>python3 src/run_module2.py --scenario urban --seeds 12</code>	trust under 5-behaviour mixed attack, 0–40 %	12	<code>results/module2_urban.json</code>
8	<code>python3 src/run_module2.py --scenario urban --seeds 10 --ablate</code>	credibility weighting on/off, two attack mixes	10	console
9	<code>python3 src/run_priority.py --scenario {urban,highway} --seeds 12 --bypass-cost</code>	cost of the aggregation bypass, no attackers	12	<code>results/priority_bypass_{scen}.json</code>
10	<code>python3 src/run_priority.py --scenario {urban,highway} --seeds 12</code>	false-priority attack × 3 defences × 6 attacker fractions	12	<code>results/priority_defence_{scen}.json</code>
11	<code>python3 src/run_priority.py --scenario {urban,highway} --seeds 12 --sweep-greed</code>	attacker greed sweep at 20 % attackers	12	<code>results/priority_greed_{scen}.json</code>
12	<code>python3 src/crypto_bench.py --seconds 3</code>	ECDSA/HMAC sizes, timings, energy sensitivity	—	<code>results/crypto_bench.json</code>
13	<code>python3 src/verify_table4.py</code>	the three-argument Table 4 refutation	10	console
14	<code>python3 src/verify.py</code> , <code>python3 src/verify_fit.py</code>	original reproducibility audit; frame-count fit	—	console
15	<code>python3 src/regression_gate.py /tmp/now.json then --diff results/regression_reference.json /tmp/now.json</code>	the invariant — 55 configurations	fixed	<code>/tmp/now.json</code>

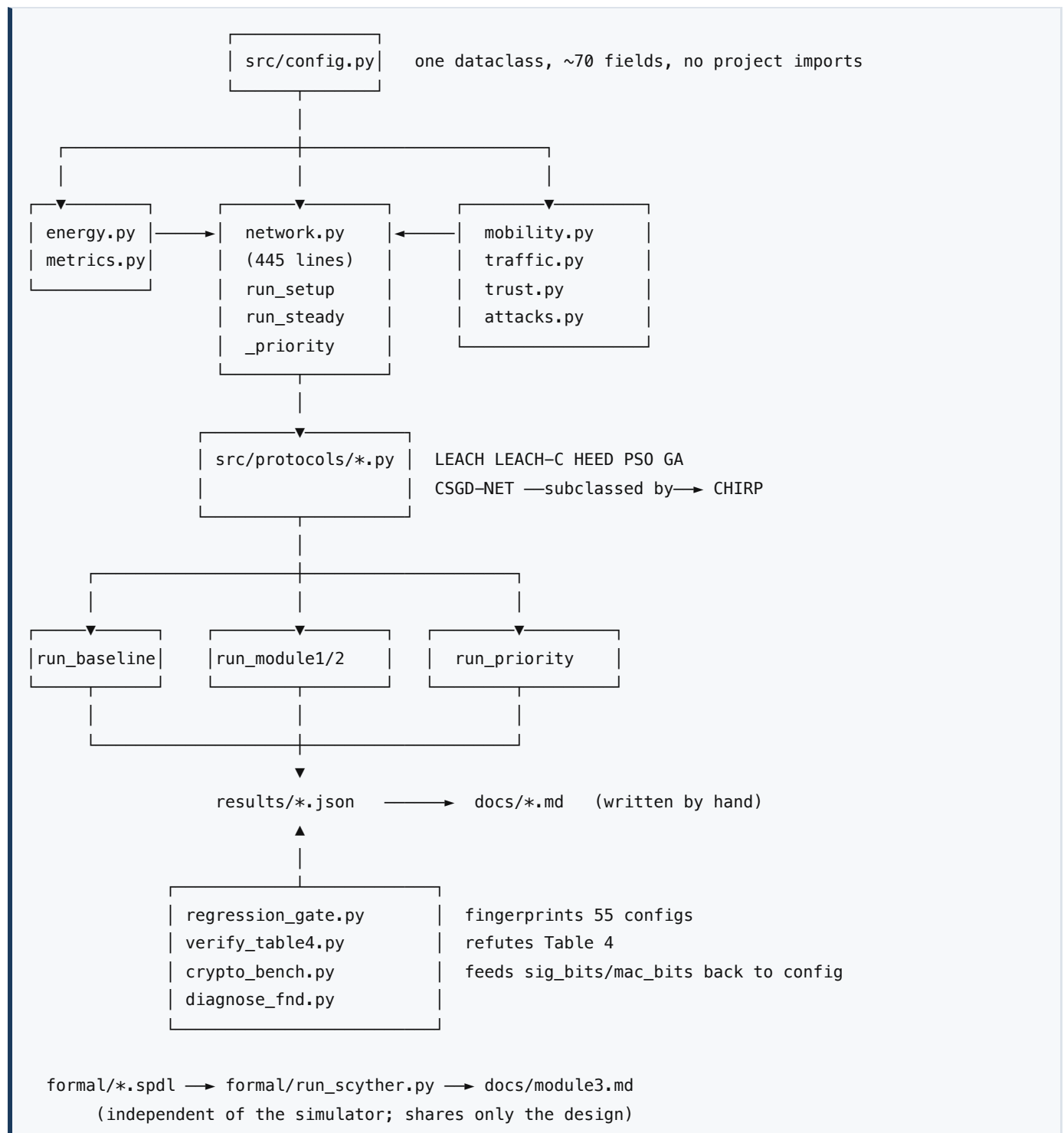
#	command	what it runs	seeds	output
16	SCYTHON=/path/to/scyther-mac python3 formal/run_scyther.py	5 Scyther models vs expected set	—	console, exit code

Full reproduction of both scenarios:

```
for s in urban highway; do
  python3 src/run_priority.py --scenario $s --seeds 12 --bypass-cost
  python3 src/run_priority.py --scenario $s --seeds 12
  python3 src/run_priority.py --scenario $s --seeds 12 --sweep-greed
done
```

Scyther is not in the repo (build artifact). v1.3.0 from github.com/cascremers/scyther ships a **native macOS arm64** binary at `Scyther/scyther-mac`.

PART 27 — FILE / DATA MAP AND INFORMATION FLOW



27.1 The one feedback loop

crypto_bench.py → measured sizes → **config.py defaults** → run_priority.py → new results/priority_*.json → updated docs/priority.md. This is the only place where a measurement tool writes back into the model's configuration, and it is why the Module 5 tables had to be regenerated when Module 3b landed.

27.2 Documentation map

file	covers
CLAUDE.md	orientation, the regression invariant, deliberate conventions, findings a fresh reader would re-derive
README.md	project summary, gaps table, module list, status, Table 4 reproduction result
docs/reproducibility.md	the audit: 5 findings, incl. the Table 4 refutation and 3 withdrawn claims
docs/threat_model.md	adversary, two-layer attack taxonomy, defence mapping
docs/module1.md	CH election, ablation, 3 negative results, FND diagnosis
docs/module2.md	trust, credibility ablation, the energy free ride
docs/module3.md	Scyther: results, the cross-protocol bug, limitations
docs/module3b.md	crypto sizes, timings, decomposition, computation-energy sweep
docs/priority.md	the largest doc — 5 findings, both scenarios, all Module 5 results
docs/related_work.md	3-paper comparison, novelty verdict, scoop risk, publication

AGENTS.md is an **untracked harness mirror of CLAUDE.md** (byte-identical); it is not part of the project.

PART 28 — GIT HISTORY

HEAD `07c9aec` , branch `master` , working tree clean (one untracked file, `AGENTS.md` , the harness mirror). No remote configured — nothing has been pushed. Four commits, all 2026-08-11.

`9ee0d09` — CHIRP: secure, priority-aware clustering for VANETs

43 files, 7,856 insertions. The initial commit: the whole simulator, six baseline protocols, CHIRP Modules 1, 2 and 5, the reproduction audit, the regression gate and its reference. **Research significance:** everything up to and including the discovery that priority is a DoS surface and that only authorisation closes it, plus the Table 4 refutation. Also the commit that inadvertently captured two 3-seed smoke-test JSONs (Part 25).

`d81eed6` — Module 3a: Scyther verification of the authorisation layer

8 files, 834 insertions. Five `.spdl` models, the runner, `docs/module3.md` . **Research significance:** turns Module 5's `require_ems_auth` boolean into a verified protocol property, and contributes the project's one externally discovered bug — the cross-protocol MAC confusion (Part 18) — plus the formal statement of the fusion provenance limit (Part 19).

`f1238dc` — Module 3b: measured crypto cost, and where it actually falls

12 files, 1,176 insertions, 678 deletions. `crypto_bench.py` , the config defaults, and a full regeneration of all six Module 5 result files. **Research significance:** overturns the project's own prediction — the radio cost of crypto is negligible, the computation cost is not, and the base paper's model cannot see it. The large deletion count is the Module 5 tables being rewritten with crypto charged.

`07c9aec` — Add `CLAUDE.md` and `related_work.md` so the project is self-contained

3 files, 293 insertions. **Research significance:** no new results. It closes a documentation gap found by audit — the three-paper comparison, the novelty verdict, the scoop risk and the publication assessment existed nowhere on disk.

Shape of the history: three research commits and one hygiene commit, each self-contained and each carrying its own negative results in the message. There is no commit that only refactors.

PART 29 — CURRENT STATE

COMPLETE

- Simulator core, Heinzelman radio, TDMA steady state, setup-phase energy
- Six baseline protocols (LEACH, LEACH-C, HEED, PSO, GA, CSGD-NET) + CHIRP
- Two synthetic mobility models + static reproduction mode
- **Module 1** — mobility-aware multi-metric CH election, ablated, 3 negative results recorded
- **Module 2** — three-layer Beta-reputation trust, 5 adversary behaviours, credibility ablation, energy free-ride finding
- **Module 3a** — Scyther, 32 claims verified, 8 by-design failures, 1 real bug found and fixed
- **Module 3b** — measured crypto sizes and timings, config fed, all Module 5 tables regenerated
- **Module 5** — priority traffic, bypass, false-priority attack, three defence regimes, greed sweep; **both scenarios**
- Reproduction audit + `verify_table4.py` refutation
- Regression gate, 55 configurations, passing
- Documentation: 10 markdown files, self-contained orientation

PARTIAL

- **Result-file hygiene.** `results/module1_urban.json` and `results/module2_urban.json` contain 3-seed smoke tests, not the 20/12-seed research runs their docs report. Docs are authoritative; JSONs need regenerating.
- **Two result tables exist only in docs, not as JSON** — the Part 8.6 coverage/latency inversion (6 seeds) and the Part 22 crypto decomposition (16 seeds). Both were produced by inline scripts.

NOT STARTED

- **Module 4** — RSU-side IDS. See Part 30.
- **Figs. 11–15 regeneration** (listed in README status).
- **Report / paper draft / slides.**

BLOCKING (for publication, not for the BTP)

- Heinzelman radio at VANET distances — needs a robustness check under a realistic 802.11p path-loss model.
- Synthetic mobility — needs SUMO traces to remove an easy reviewer objection.

OPTIONAL

- Charging computation energy in the simulator (magnitude documented; would move Module 1/2 baselines).
 - `priority_budget` token bucket is implemented but defaults to unlimited (0) and is not exercised in any reported experiment.
 - ProVerif/Tamarin for the unlinkability property Scyther cannot express.
 - Revocation-rather-than-demotion, and duty rebalancing — the two remedies Module 2 identified for the energy free ride.
-

PART 30 — MODULE 4, INVESTIGATED

30.1 Intended purpose (from `README.md`)

RSU-side IDS — behavioural features → Random Forest / autoencoder, with a revoke → trust-reset → exclude feedback loop. Validated on the generated dataset and on the public VeReMi dataset.

30.2 Actual state — NOT STARTED

Verified by search, not assumption:

check	result
<code>src/run_module4.py</code>	does not exist
<code>src/ids.py</code>	does not exist
<code>docs/module4.md</code>	does not exist
<code>import sklearn</code> anywhere in <code>src/</code>	none
"VeReMi" in code	one docstring only , in <code>src/attacks.py</code> , explaining which attacks VeReMi does <i>not</i> contain
any <code>results/module4*.json</code>	none

`scikit-learn` appears in the README's install line, provisioned for this module and currently unused.

30.3 What exists that is adjacent

- `docs/threat_model.md` maps the intended Layer-B attacks (position/speed falsification, Sybil, DoS, replay) onto VeReMi Extension attacker types, and identifies one attack — **forged aggregate at a compromised CH** — as *ours only*, because VeReMi has no clustering.
- Module 2 identified the two mechanisms Module 4 was meant to supply: **revocation rather than demotion**, and **duty rebalancing**, both aimed at the energy free ride.

30.4 What would remain

Feature extraction from simulator traces; a classifier; validation on VeReMi Extension; and the revoke → trust-reset → exclude loop wired back into `TrustEngine` and `CHIRP._pool()`.

30.5 Why it is judged the least novel component

From `docs/related_work.md`, and verified against the comparison papers:

Comparison paper P2 (Sathik Raja et al., *SN Computer Science* 2026) already does RF-style machine learning on VeReMi from Kaggle — ~11,000 records, 80/20 split, reporting Sybil/replay/blackhole detection. An RF/autoencoder IDS on VeReMi is a heavily worked area with many prior entries.

Worse, P2's approach exposes exactly the gap that makes our simulator valuable and Module 4 redundant: **VeReMi never simulated routing**, so it contains no packet-dropping and no reputation-manipulation attacks. Running a classifier on it validates against a dataset that does not contain the Layer-A attacks this project actually defends against.

Conclusion recorded in `docs/related_work.md`: Module 4 is the candidate to cut if time is short. Nothing else in the project depends on it.

PART 31 — NOVELTY AND RELATED WORK

Verified against docs/related_work.md and the repository.

31.1 The three comparison papers

	P1 Satheshkumar et al.	P2 Sathik Raja et al.	P3 Jaswani et al.
venue	<i>Iranian J. Sci. Tech., Trans. EE</i> 2026	<i>SN Computer Science</i> 2026, 7:563	<i>Security and Privacy</i> 2026, 9:e70241
DOI	10.1007/s40998-026-01150-y	10.1007/s42979-026-05154-7	10.1002/spy2.70241
scheme	FCKOA-TA-AROA: fuzzy C-means + Kookaburra CH + Aqua Raptor routing + homomorphic encryption	BCS-DMCO: Binary Cat Swarm CH + Markov link prediction + AES-128	AODV + trust + ECC signcryption
architecture	cluster + multi-hop route	cluster + route	no clustering
trust	linear weighted sum (Eq. 29)	direct + credibility-weighted indirect	+0.001/packet, drops>100 → 0
attacks	Sybil + DoS, 5 % attackers	Sybil/replay/blackhole as tabular ML on VeReMi	blackhole only
mobility	random waypoint, MATLAB, 500 nodes	unspecified synthetic	SUMO + OSM + NS-2.35 , 50 nodes
statistics	bar charts, single runs	± values, no test	single runs
ablation / negatives	none	none	none

P3 is a different problem — no clustering, no CH election, no energy model. Not a competitor.

P2 overlaps in one place that matters: its Eqs. 17–18 are a feedback-credibility-weighted indirect trust — **the same idea as our Module 2 layer 3. We cannot claim credibility weighting as novel.**

P1 is the real overlap. Clustering + metaheuristic CH election with trust in the objective + trust-aware routing + encryption is our four-module stack, one-for-one. **If the abstract reads "we add trust and crypto to a metaheuristic clustering protocol for VANETs", a reviewer hands you P1.**

31.2 The scoop risk — the one to actually worry about

The base paper's own §5: "*As future direction, we are now working on an optimized version of the present solution considering the node mobility constraint and thus the network topology frequent change.*"

The original authors have announced they are building the mobility-aware version of CSGD-NET — which is Module 1.

Mitigation: Module 1 must not be the headline. Frame it as infrastructure — "*we first make CSGD-NET mobility-capable, as its authors indicate is needed*" — and lead with the security, priority and audit work. That framing survives even if their paper lands first; it becomes motivation rather than competition.

31.3 Honest classification of our own work

Not novel — engineering / reimplementaion: - Trust as a term in a metaheuristic CH fitness (P1, P2, a decade of WSN work) - Beta reputation (Jøsang & Ismail 2002) - Credibility-weighted recommendations (P2 has it) - ECC pseudonyms with conditional privacy (P3 and a large literature) - An RF/autoencoder IDS on VeReMi (P2 is already there) → **Module 4**

Extension: - Mobility on top of CSGD-NET (and the base authors are doing this too) - The six-term objective, of which the ablation shows only two terms carry

Genuinely contributory — five things, none of them a mechanism: 1. **Trust-based demotion pays the attacker** — $2.0\text{--}2.4\times$ residual energy. Corollary: *a trust defence cannot be validated on energy metrics*, which indicts P1's "42 J" and P2's "energy balancing index" headlines. 2. **Priority is a DoS surface**, and a behavioural detector provably cannot separate a restrained liar from a real ambulance. 3. **Coverage and deadline latency are in direct opposition** — measured across 7 protocols $\times$ 2 scenarios. 4. **Crypto's cost is computation, not bytes**, and the base paper's radio-only model is structurally blind to it. 5. **The reproduction audit** — Eq. 9 under-determined; Table 4's LEACH-C and CSGD-NET columns unreachable.

Negative results (a category the comparison papers have none of): three in Module 1, the coverage/latency inversion, detection $\neq$ mitigation, and the fusion-provenance limit.

Methodological: the controlled-optimiser comparison (CHIRP subclasses CSGDNet, so a difference is attributable to the objective), the regression gate, and 12–30 seeds with significance tests throughout.

PART 32 — PUBLICATION AND BTP STRATEGY

32.1 Two possible papers

Paper A — "CHIRP: a secure trust-aware cuckoo clustering protocol for VANETs." Publishable in the tier P1/P2 occupy. Low risk, **low value**, competes head-on with P1. Not recommended as the primary framing.

Paper B — the audit, the energy free ride, and priority as an attack surface. Harder to place, worth more, and **it is what has actually been built**.

venue	fit
<i>Vehicular Communications</i> (Elsevier)	strong — VANET-specific, takes critical/measurement work
<i>Simulation Modelling Practice and Theory</i>	strong — "published model is under-determined" is its remit
IEEE VNC, short paper	strong — fast, right audience, best route to something citable before the deadline
<i>Ad Hoc Networks / Computer Networks</i>	moderate — wants more than one novel result
ACM TOMACS	moderate — prestigious, slow, would want the simulator as the artefact

32.2 Strongest and weakest

- **Strongest:** the priority-as-DoS chain — bypass necessary → bypass is a finite resource → behavioural detection provably insufficient → authorisation necessary and verified. It is a complete argument with measurement *and* proof at every step, and nothing in P1/P2/P3 resembles it.
- **Runner-up:** the energy free ride, because it invalidates a metric the whole comparison literature reports.
- **Weakest:** Module 1 as a novelty claim (saturated, and the base authors are doing it). **Module 4** (not started, and P2 got there first).

32.3 What can and cannot be cut

Can be cut: Module 4 entirely. Nothing depends on it. **Must not be cut:** the regression gate (it is what makes the comparisons trustworthy), the negative results (they are a differentiator), and the reproduction audit (it justifies not using Table 4 as a baseline).

32.4 Remaining blockers

1. ~~LEACH-C/CSGD-NET reproduction gap~~ — **resolved**, diagnosed as a defect in the published table.
2. **Heinzelman radio at VANET distances** — needs a robustness check under 802.11p path loss showing the free-ride and intra-term results survive.
3. **Synthetic mobility** — SUMO traces; P3 already does this.
4. **Computation energy measured but not charged** — a reviewer may ask.

32.5 Timing

Journal review here is 4–12 months to first decision. **IEEE VNC's short-paper track is the fastest route to something citable before graduation**; keep the journal version for later.

32.6 One thing to handle carefully

`docs/reproducibility.md` findings 1 and 3 concern the published Table 4. **Keep them exactly as written — an observation about published numbers with a stated methodological consequence (the table cannot be a baseline), never an allegation about how they were produced.** Some venues will ask for it to be cut; the paper does not depend on it, so cut it if asked.

PART 33 — LIMITATIONS

#	limitation	why it exists	effect on claims	blocking?	future work
1	Synthetic mobility	no SUMO dependency, self-contained	cannot claim real-world validation; "highway"/"urban" are stylised	not for BTP; yes for publication	replay SUMO/OSM traces
2	No channel model — no contention, collisions, interference, fading	out of scope of the base paper's model	no throughput/jitter/flooding-DoS claims possible; slot exhaustion is modelled honestly	no	integrate with ns-3/Veins
3	Delay in TDMA slots, never ms	no time model below the round	latency results are relative, not absolute	no — it is stated everywhere	add a MAC/queueing layer
4	Heinzelman radio at VANET distances	inherited from the base paper	d ⁴ at 100–150 m makes absolute lifetimes unrealistic; trends comparable since all protocols charged identically	yes for publication	802.11p/WINNER+ path loss as robustness check
5	Computation energy not charged	would move Module 1/2 baselines	Module 5's cost of priority remains a lower bound	no — documented	charge it, re-baseline everything
6	OBU slowdown and CPU power are assumptions	no OBU hardware available	the $\sim 15\times$ figure is an extrapolation, not a measurement	no — labelled	measure on real OBU or automotive HSM
7	Only two hops — member $\rightarrow$ CH $\rightarrow$ RSU	inherited	no multi-hop route claims; not directly comparable to P1/P2	no	out of scope
8	No downlink	inherited	an ambulance cannot <i>notify</i> other vehicles, only the RSU	no	a genuine extension opportunity
9	Scyther bounded at <code>--max-runs=4</code>	search cost	"no attack within 4 concurrent runs"	no	raise the bound
10	Unlinkability not verified	Scyther proves secrecy/authentication only	conditional privacy is claimed as a design goal, not proven	no — stated	ProVerif observational equivalence
11	TA vetting is an assumption	not a message-exchange property	we verify the role cannot be forged/replayed, not that issuance policy is correct	no — stated	out of scope
12	Table 4 unusable as a baseline	defect in the published table	all comparisons use our re-implementations	no — resolved	—
13	Two urban JSONs are 3-seed smoke tests	overwritten during verification, then committed	docs are authoritative; JSONs inconsistent	no — but should be fixed	re-run at 20/12 seeds
14	Two result tables exist only in docs	produced by inline scripts	not reproducible from a committed runner	no	promote to runners
15	Emergency traffic is 0.84 % of readings	5 % EMS $\times$ 4/22 messages	crypto/bypass costs are small in absolute terms; would scale with heavier load	no	sweep <code>ems_frac</code>
16	Class 1 (per-vehicle safety events) not modelled	two classes suffice to show the mechanism	no claims about general safety traffic	no	add a third class
17	Module 4 not built	time; judged least novel	no IDS claims; the free-ride remedies are unimplemented	no	cut or build

#	limitation	why it exists	effect on claims	blocking?	future work
18	Related-work search was targeted, not systematic	~8 queries	"no duplicate found" is weaker than a Scopus sweep	yes before submission	systematic review

PART 34 — HOW TO EXPLAIN THIS TO YOUR GUIDE

Speakable answers. Technically accurate, no jargon you would have to unpack.

"What problem are you solving?"

Vehicle networks save battery by grouping cars, having one car collect everyone's data and squash it into a single message. That squashing is the whole energy saving. The problem is that to squash, the group leader has to wait for everyone to speak — so every message inherits the delay of the whole group. That's fine for routine sensor data. It is not fine for an ambulance.

"Why is the base protocol insufficient?"

Three things. It assumes a static topology — in a *vehicular* network, which its own discussion section admits. It has no security at all: its cluster-head election reads only self-reported energy and distance, so a node that lies gets elected and can drop everything. And it has no notion of an urgent message — the words "priority", "emergency" and "deadline" do not appear in the paper.

"What did you change?"

Four things. Added real mobility and a better cluster-head objective — the base paper's formula has no term for how far members are from their own head, and our ablation shows that's the second most important term. Added a trust engine so a misbehaving node can't get elected. Added an emergency traffic class with a deadline and a fast path. And added cryptography, which we verified formally.

"What did you discover?"

Several things we didn't expect. The biggest: being *good* at clustering makes you *worse* at emergency latency, and we measured that across seven protocols. Also that our trust engine detected an attacker 94 % of the time and made no difference to the outcome. And that the base paper's own results table contains numbers its own equations can't produce.

"Why did you add priority?"

Because an ambulance's message can't wait for the group to finish, and can't be averaged into a fused packet with ten temperature readings. We measured it: without a fast path, emergency messages miss their deadline 87–99 % of the time.

"Why did priority create a security problem?"

Because the fast path is a *reserved* resource — a limited number of slots per group per round. And the group leader has no way to check whether the car claiming to be an ambulance really is one. So a liar takes the ambulance's slot. With 20 % of cars lying, real ambulances miss 44 % of deadlines and 11 % of the whole network's battery goes to carrying fake-urgent messages.

"Why isn't trust enough?"

Two reasons. First, our initial trust defence detected the attacker at 94 % and changed nothing — because its only power was to stop a node being elected *leader*, and this attack doesn't need to be a leader. Detection isn't mitigation unless it's wired to something. Second, even after we fixed that, a patient attacker who claims priority at exactly the rate a real ambulance does is *invisible* to any behaviour-based test — we measured detection at 0.008, below its own false-alarm rate — and still does about half the damage.

"Why do you need authorisation?"

Because the thing that distinguishes an ambulance from a liar isn't behaviour — it's whether it holds a credential. A signed certificate from the authority saying "this vehicle has the emergency role". With that, the attack goes to zero at every attacker level, in both scenarios. And unlike trust, it costs nothing when there's no attack — no false alarms.

"Why Scyther?"

Because that whole result rests on one assumption: that a group leader grants a reserved slot *only* to a genuinely certified emergency vehicle, and never to a replayed message. In the simulator that's just a boolean flag. Scyther is where we proved it's a real property of the protocol rather than an assumption. And it earned its place — it found a real bug in our design that the simulator physically could not have found.

"Why does fusion create a provenance problem?"

Because fusion throws information away. When the leader squashes ten readings into one number, nothing in that number lets the roadside unit check it's a faithful summary. The message authentication code proves *who sent it*, not *whether it's true*. So a compromised leader can fabricate the fused value and no MAC will catch it. We built a formal model specifically to demonstrate that — it's designed to fail, and the failure is the result.

"What does the cryptography cost?"

On the radio, almost nothing — under 1.7 % of network energy, and the per-message signature is statistically undetectable. But that's the wrong place to look. Verification is 2.9 times more expensive than signing, and it's the operation we do most. On plausible vehicle hardware, verifying one signature costs about 15 times the radio energy of transmitting it. The base paper explicitly assumes computation is free, so its model literally cannot see this.

"What happens on urban versus highway?"

Every finding replicates, which is the main point. Two things only show up in comparison. The cost of priority is set by RSU density, not traffic — highway RSUs are close enough that transmission costs distance-squared, urban ones are far enough to cost distance-to-the-fourth, which is three times more expensive for the same feature. And trust's false-alarm cost only appears in urban, because highway networks live twice as long and the trust engine has time to resolve honest nodes.

"What is actually novel?"

Honestly, not the mechanisms — trust in a fitness function and Beta reputation are standard. What's novel is five findings: that trust-based demotion actually rewards the attacker with more energy, that priority is a denial-of-service surface, that coverage and deadline latency are in direct opposition, that crypto's cost is computational rather than transmission, and that the base paper's results table isn't reproducible from its own equations.

"What are the limitations?"

Mobility is synthetic, not SUMO traces. There's no radio channel, so no collisions and no throughput claims. Delay is in TDMA slots, not milliseconds, because there's no time model below the round — I'd rather report a defensible unit than invent one. And the computation-energy figure is an extrapolation from a laptop, not a measurement on vehicle hardware.

"What remains?"

The RSU-side intrusion detector, Module 4 — which I'd argue is the piece to cut if time is short, because a 2026 paper already does exactly that on the same public dataset. Then regenerating the figures and writing up.

PART 35 — FIVE-MINUTE EXPLANATION

This project starts from a 2026 *Cluster Computing* paper called CSGD-NET. It makes vehicle networks energy-efficient by clustering: cars form groups, one car per group is elected leader, everyone sends their data to the leader, and the leader **fuses** it all into a single packet for the roadside unit. Fusion is where the energy saving comes from — one transmission leaves the group instead of ten. The leader is elected by Cuckoo Search, scoring candidates on residual battery and distance to the roadside unit.

We reimplemented it faithfully, alongside five comparison protocols, and then extended it. But the first thing we did was try to *reproduce* it, and that turned up something. The paper's headline results table has six columns. We reproduce the LEACH column to an RMSE of 0.005 — but not LEACH-C or CSGD-NET. We proved why: under the paper's own equations, the electronics term of the radio model is paid identically no matter which nodes are leaders, so the only thing a clustering algorithm can influence is the amplifier term — about 0.002 joules per node. The spread between the paper's columns is 20 to 44 times that. Even a *perfect* clustering algorithm, with zero amplifier energy, falls short of the published LEACH-C column. And two of its columns show energy drain *increasing* between intervals, which is impossible when nodes only die. The table isn't a usable calibration target, so all our comparisons use our own re-implementations.

Then the extensions. Module 1 adds mobility and a six-term leader-election objective, including the intra-cluster distance term the original omits — our ablation shows that's the second most important term. Module 2 adds a three-layer Beta-reputation trust engine, tested against five attack behaviours. That produced our first surprising result: **trust-based demotion actually rewards the attacker**. Excluding a misbehaving node from the leader role spares it the most expensive job in the network, so detected attackers end with 2 to 2.4 times the battery of honest nodes. Which means a security defence **cannot be validated on energy metrics** — and that indicts the headline metrics of most papers in this space.

Module 5 is the core contribution. Fusion forces every reading to wait for the whole TDMA frame, so we asked: what does that do to emergency traffic? We built a delay model in TDMA slots — not milliseconds, because there's no channel and a millisecond figure would be invented — and found that **deadline misses track orphan rate almost perfectly inversely**, across seven protocols and two scenarios. Protocols that are *good* at coverage are *bad* at deadlines, because isolated nodes skip fusion and are the fastest path in the network. Our own protocol has the worst emergency latency precisely because it has the best coverage.

So we gave emergency traffic a reserved slot that bypasses fusion. That takes deadline misses to zero for under 2 % of network energy. But the reservation is finite, and a cluster head can't authenticate a claim on it. So we built the attack: a vehicle that falsely claims emergency status. At 20 % attackers, real ambulances miss 44 % of deadlines and 11 % of the network's entire energy budget goes to carrying fake-urgent messages. The attacker spends nothing extra — it **externalises** the cost onto honest cluster heads.

Our first defence used the existing trust engine. It detected the attacker at 94 % and changed the outcome **not at all** — because the trust gate's only lever was leader eligibility, and this attack doesn't require being a leader. **Detection is not mitigation unless you name the enforcement path**. We fixed that, which halves the damage. But then we swept attacker greed and found the real limit: an attacker asserting priority at exactly the rate a genuine ambulance uses is invisible — detection 0.008, below its own false-positive rate — and still takes half the maximum damage. No rate-based test can ever separate them, because their behaviour is identical by construction.

Only a cryptographic role attribute works, and it gives **zero deadline misses at every attacker fraction in both scenarios**, with no false-positive cost. We verified that protocol in Scyther: 32 claims verified, and the load-bearing one proves a head grants a reserved slot only to a genuinely certified vehicle and never to a replay. Scyther also found a **real bug** — our member-reading and fused-aggregate MACs had identical structure, so one could be replayed as the other. Invisible in the simulator, where those are different Python variables. Fixed with domain-separation tags.

Finally we measured what the crypto costs. On the radio: almost nothing, and the per-message signature is statistically undetectable. But **verification is 2.9× the cost of signing, and it's what we do most** — on plausible vehicle hardware it costs roughly 15× the radio energy of the signature it checks. The base paper explicitly assumes computation is free, so its model is structurally incapable of showing that.

PART 36 — FIFTEEN-MINUTE DEEP EXPLANATION

(1) Setting — 2 min

Base paper: Sellami, Mchergui & Alaya, *Cluster Computing* 29:16 (2026), CSGD-NET. Cuckoo Search with Lévy flights plus a Gaussian walk elects a cluster-head set; fitness is $f = E / (E_0 \cdot \text{dist}(\text{CH}, \text{RSU}))$. 100 nodes, 50×50 m, static, Heinzelman radio, one RSU at the centre, TDMA within clusters, isolated vehicles report direct to the RSU.

Note three structural facts about it, all of which matter later: **there is no cluster-formation protocol** (membership is never specified), **there is no multi-hop routing** despite the title, and **it explicitly assumes computation costs no energy**.

(2) Reproduction and audit — 3 min

We reimplement CSGD-NET plus LEACH, LEACH-C, HEED, PSO and GA on a shared radio model so all are charged identically. One unstated parameter — TDMA frames per round — is fitted to Table 4's LEACH column at 22, giving RMSE 0.005.

LEACH-C (0.049) and CSGD-NET (0.046) do not reproduce. We first recorded that as our defect and then **refuted our own claim** with three arguments:

1. The electronics term $(2(n - n_{\text{ch}}) + n_{\text{ch}}) \cdot k \cdot E_{\text{elec}}$ is protocol-independent, so the amplifier is the entire budget a clustering algorithm controls — measured at 0.00130–0.00204 J/node at round 10.
2. Table 4's cross-protocol spread is 0.04018 (three protocols) — **19.7×** that budget; across all six columns, 44.2×. And a **zero-amplifier** LEACH-C still falls 0.0108 J/node short of its column.
3. Drain per round = alive × per-node cost, both non-increasing, so the drop between sample points cannot grow. The paper's LEACH-C rises **+56.5 %** and CSGD-NET **+60.2 %**.

The only internally-consistent column is LEACH — the only one we reproduce. Six alternative readings of "Remaining energy" and both round alignments were tested; none closes it. A ~300 m field *would*, but Table 3 says 50×50 m, so adopting it would be curve-fitting against a published parameter. **We changed no simulator code.**

(3) Architecture and the regression gate — 2 min

Round = mobility step → CH election → setup phase (ADV/JOIN, which the base paper charges nothing for and we do) → 22 TDMA frames → trust update.

`results/regression_reference.json` fingerprints **55 configurations** — three scenarios × seven protocols × seeds, plus the trust engine under attack — hashing the raw float64 bytes of the energy vector, the PDR counters, the orphan and intra traces and the trust scores. **Invariant: with `ems_frac = 0` and `sig_bits = mac_bits = 0`, every pre-Module-5 result must be bit-identical.** It holds because `_priority_phase` returns early when no traffic model is attached, and the traffic model is attached by the *runner*, so `Network` never draws from its own RNG for it. This is what makes every comparative claim trustworthy: each is a difference against a baseline, and the gate proves the baseline didn't move.

(4) Modules 1 and 2 — 3 min

Module 1. Six-term objective; two model corrections (setup energy; fitness scored in `tx_energy` rather than distance, because the radio has a knee at $d_0 = 87.71$ m and a head at 150 m costs ~9× one at 88 m). Highway results: FND +46 % over CSGD-NET, orphan rate −98.3 %, all five metrics significant. The ablation shows only **energy** and **intra** carry — and intra is the term Eq. 9 omits.

Three negative results, all kept as disabled code paths with the evidence: stability-triggered re-clustering (the head role costs **311×** a re-election), LET-aware association (monotonically worse), and a survival-prediction term (the optimiser raised survival by *orphaning* members).

Module 2. Three-layer Beta reputation: direct watchdog trust, decay with a 3× asymmetric penalty, credibility-weighted recommendations. PDR at 20 % attackers 0.957 vs 0.934, $p = 0.0018$. Credibility weighting cuts the false-positive rate 12–49 %, but does *not* uniformly improve detection — and the doc says so.

The finding: detected attackers end with **2.0–2.4× the residual energy** of honest nodes, because demotion spares them the 88 mJ/round head role.

(5) Module 5, the core — 3 min

Delay model in **TDMA slots**: aggregated = cluster size + 1, bypassed = 2, orphan = 1. Deadline 4 slots, applied to class-2 only; a dropped emergency message counts as a miss.

Finding 1 — coverage vs latency. Deadline-miss tracks orphan rate inversely across 7 protocols × 2 scenarios. HEED strands 47 % of nodes and has the *best* emergency latency; CHIRP strands 0.1 % on the highway and has the worst (0.958).

Finding 2 — the bypass. Emergency readings forwarded verbatim rather than fused. Deadline misses → 0.000, p95 → 2.0 slots, for +1.63 % urban / +0.77 % highway including crypto.

Finding 3 — the attack. `falsepriority`: assert on all 22 readings instead of 4. One attacker per cluster exactly saturates the reservation. At 5 % attackers, **83 % of the reservation's energy already serves liars**. At 20 %: deadline-miss 0.437 urban / 0.472 highway, drain 11.26 % / 6.01 %.

Finding 4 — detection ≠ mitigation. First trust defence: detection 0.925, outcome 0.452 → 0.463. Unchanged, because the gate only controlled CH eligibility. Fixed with a slot-grant gate; halves the damage.

Finding 5 — trust bounds, authorisation stops. At greed 0.50 the attacker is invisible (detection 0.008–0.025, at or below FPR), takes ~47 % of maximum deadline damage and ~81 % of maximum drain, and trust makes it marginally worse. Authorisation: **0.000 at every attacker fraction, both scenarios, p = 5.1e-06**.

(6) Formal verification — 2 min

Scyther v1.3.0, `--max-runs=4`, **32 claims verified, 8 by design**.

Load-bearing: `priority.spdl claim_C4(C, Nisynch)` — Ok. Over three roles, Nisynch requires every preceding event in every role to match, so a head's slot grant implies a real TA issuance to that vehicle and a fresh assertion by it.

Scyther found a real bug. Member readings and fused aggregates were both `{sender, value, nonce, receiver}k(sender, receiver)` — structurally identical, so a head's aggregate could be replayed as a member's reading. Fixed with domain-separation tags; the suite then verifies identically whether checked file-by-file or concatenated.

Invisible in the simulator, where message types are different Python variables.

`fusion-limit.spdl` fails **by design**: the RSU cannot authenticate a member through a fused packet, because the fused value isn't a recoverable function of the readings. So a compromised head can fabricate it and no aggregate MAC detects that. Which gives the bypass a **second justification unrelated to latency**: it is the only path giving the RSU end-to-end authentication.

Six claims in `priority.spdl` fail — **not a vulnerability**. The TA never learns which head the vehicle will later talk to, so Weakagree's partnering relation can't hold across three roles. Confirmed by deleting the TA role and watching all six verify (`priority-assert.spdl`, 8/8).

(7) Crypto cost and limitations — 2 min

Measured: 64 B signature + 8 B cert digest = `sig_bits 576`; 16 B MAC = `mac_bits 128`; 122 B certificate itemised. Sign 33.4 μs, **verify 97.4 μs — 2.9×**, and verify is what we do most.

Decomposed with paired Wilcoxon over 16 seeds: whole crypto layer + bypass costs **+1.63 % urban, +0.77 % highway**; the per-message signature is **statistically undetectable** (p = 0.94 highway, 0.058 urban). Say "we could not detect a cost", not "it is free".

Computation is where the cost is — and this is an *extrapolation*, clearly labelled: at 25× OBU slowdown and 0.5 W, verification is 1.22 mJ against 0.078 mJ to transmit what it verifies, ~15×. Break-even at 1 W is 78 μs, already exceeded on this laptop.

Unanticipated interaction: charging real signature bits made the attack *worse* for victims (drain 10.41 → 11.26 % urban), because fake-urgent messages now carry signatures the honest head must relay. Authorisation removes it by refusing the claim before the forward.

Limitations, stated plainly: synthetic mobility, no channel, slots not milliseconds, Heinzelman at VANET distances, computation energy not charged, unlinkability not verified, Module 4 not built.

PART 37 — EXPECTED EXAMINER QUESTIONS

Q. Why CHIRP — why not just use LEACH? *Short:* LEACH has no mobility, no security, and its random self-election ignores position entirely. *Deeper:* the point isn't that CHIRP beats LEACH — on the highway HEED (27.4) and LEACH-C (25.2) both beat CHIRP's FND of 23.0, and `docs/module1.md` says so. The point is the *controlled comparison*: CHIRP subclasses CSGD-NET and inherits the Cuckoo engine byte-for-byte, so any measured difference is attributable to the objective rather than to a different optimiser. *Evidence:* `src/protocols/chirp.py` class declaration; `results/module1_highway.json`.

Q. Why does better clustering hurt emergency latency? *Short:* orphans skip fusion, so they're the fastest path. Good clustering removes them. *Deeper:* a clustered reading can't leave until the head has heard every member, because the head is fusing them all. So delay $\approx$ cluster size + 1. Absorbing orphans into clusters makes clusters bigger, frames longer, and every reading later — and removes the accidental fast path. *Evidence:* Part 8.6 table, 7 protocols $\times$ 2 scenarios; `docs/priority.md` Finding 1.

Q. Why bypass fusion rather than just making clusters smaller? *Short:* smaller clusters mean more heads, and the head role is the most expensive job in the network. *Deeper:* head cost is 88 mJ/round against 11.6 for a member. And it wouldn't fix the problem — it would trade emergency latency against network lifetime rather than resolving the conflict. The bypass decouples them: emergency delay becomes constant at 2 slots regardless of cluster size. *Evidence:* `src/diagnose_fnd.py` cost table; Part 10.3.

Q. Why does the bypass create a DoS opportunity? *Short:* it's a reserved, finite resource that the head cannot authenticate. *Deeper:* capacity is `emergency_slots  $\times$  packets_per_round` = 22 per cluster per round. A genuine ambulance asserts on 4 of its 22 readings; an attacker on all 22. One attacker per cluster exactly saturates it. And without authentication the head must share proportionally, so the liar takes a share the ambulance needed. *Evidence:* `Network._priority_phase`; Part 11.

Q. Why is trust insufficient? *Short:* two separate reasons — a wiring failure, then a fundamental one. *Deeper:* first, our trust gate controlled only CH eligibility, and this attack doesn't require being a head: detection 0.925, outcome unchanged (0.452 $\rightarrow$ 0.463). Second, even wired correctly, an attacker asserting at the genuine ambulance rate is behaviourally identical to an ambulance — detection 0.008, below its own false-positive rate — and still takes $\sim$ 48 % of maximum damage. *Evidence:* Part 12.2; `results/priority_greed_*.json`.

Q. Then what exactly does authorisation give you? *Short:* zero deadline misses at every attacker fraction, in both scenarios, with no false-positive cost. *Deeper:* it changes the question from "has this vehicle behaved like an ambulance?" — which a patient attacker answers correctly — to "is this vehicle an ambulance?", which it cannot. False demand is refused before the reservation is shared, so genuine demand is never diluted and no energy is externalised. *Evidence:* `results/priority_defence_*.json`, all `auth` rows 0.000.

Q. What does `p = 5.1e-06` actually tell me? *Short:* complete separation between the two arms, not effect size. *Deeper:* with 12 seeds per arm where `auth` is 0.000 in every seed and undefended is positive in every seed, the ranks separate completely, so the Mann-Whitney statistic takes its extreme value — that `p` is the minimum attainable at `n = 12`. The effect size is the deadline-miss difference itself, 0.437 $\rightarrow$ 0.000. *Evidence:* `src/run_priority.py` significance block.

Q. Why Scyther, and what did it actually prove? *Short:* it proved the one assumption all the authorisation results rest on, and it found a bug. *Deeper:* `claim_C4(C, Nisynch)` — over a three-role protocol, Nisynch requires every preceding event in every role to match, so a head's slot grant implies a real TA issuance to that vehicle and a fresh assertion. In the simulator that's a boolean; here it's a proved property. *Evidence:* `formal/priority.spdl`; `formal/run_scyther.py` output.

Q. What did Scyther discover that you didn't already know? *Short:* a cross-protocol attack between our two MAC formats. *Deeper:* member readings and fused aggregates were both `{sender, value, nonce, receiver}k(sender, receiver)`. A head's aggregate to the RSU could be accepted as a member's reading, and vice versa. Signature of the failure: Alive and Weakagree passed while Niagree and Nisynch failed — right peer, wrong message meaning. The simulator physically cannot represent this, because those are different Python variables reached by different code paths. *Evidence:* Part 18 trace; `formal/aggregate.spdl` header comment.

Q. Why does fusion lose provenance? *Short:* fusion is lossy and non-invertible, and a MAC authenticates the sender, not the truth of the value. *Deeper:* `fusion-limit.spdl` is built to fail and does — R's Niagree and Nisynch both fail. So a compromised head can fabricate the fused value undetectably. That gives the class-2 bypass a security justification independent of latency: it's the only path where the vehicle's own signature survives to the RSU. *Evidence:* `formal/fusion-limit.spdl`; Part 19.

Q. Six of your Scyther claims fail. Is your protocol broken? *Short:* no — it's a modelling limitation, and we proved that by experiment. *Deeper:* the TA never learns which cluster head the vehicle will later talk to, because registration happens offline and once. Weakagree over three roles asks whether *every* role is running with the claimer, and the TA isn't running with any head. Delete the TA role and all six verify — that's `priority-assert.spdl`, 8/8. We tested two wrong explanations first and discarded both. *Evidence:* Part 20; `formal/priority-assert.spdl`.

Q. Why is the crypto cost mostly computational? *Short:* 576 extra bits are nothing next to a 6400-bit payload plus a fixed electronics term; a signature verification is not. *Deeper:* verify measures 97.4 μ s and is $2.9\times$ signing. At $25\times$ OBU slowdown and 0.5 W that's 1.22 mJ, against 0.078 mJ to transmit the signature — about $15\times$. Break-even at 1 W is 78 μ s, already exceeded on this laptop. **The slowdown and power are assumptions and are swept, not asserted.** *Evidence:* `results/crypto_bench.json`; `docs/module3b.md`.

Q. So is the signature free? *Short:* no — say we couldn't detect it. *Deeper:* $p = 0.94$ highway, 0.058 urban at 16 seeds. Absence of a detected effect isn't proof of no effect; more seeds might resolve the urban $+0.71\%$. *Evidence:* Part 22.

Q. Why don't LEACH-C and CSGD-NET reproduce? *Short:* because those published columns can't be produced by the paper's own equations at its own parameters. *Deeper:* the electronics term is protocol-independent, so the amplifier — 0.002 J/node — is the entire budget clustering controls; Table 4's spread is $20\text{--}44\times$ that. A zero-amplifier LEACH-C still falls 0.0108 short. And two columns show drain *rising* between intervals, which `alive  $\times$  cost` can't do. LEACH is the only internally consistent column, and it's the only one we reproduce. *Evidence:* `python3 src/verify_table4.py`.

Q. Couldn't you just change the field size to match? *Short:* yes, ~ 300 m works — and that would be curve-fitting. *Deeper:* Table 3 explicitly states 50×50 m with nothing contradicting it. We record the 300 m result as a *hypothesis* about how Table 4 might have been produced, and deliberately made no code change. Never allege fabrication — the linearity and non-monotonicity are facts; the cause isn't knowable from outside. *Evidence:* Part 16.7; `docs/reproducibility.md` finding 1.

Q. Why synthetic mobility rather than SUMO? *Short:* self-containment — the project carries no SUMO dependency. *Deeper:* it's a genuine limitation and one of the two publication blockers. Both scenarios differ enough in geometry, RSU density and speed to give real cross-scenario evidence, and two findings appear only in comparison. But it is **not** real-world validation and we don't claim it is. Comparison paper P3 does use SUMO + OSM + NS-2.35. *Evidence:* `src/mobility.py`; Part 15.4.

Q. Why report delay in slots instead of milliseconds? *Short:* because there's no channel and no time model below the round, so milliseconds would be invented. *Deeper:* a TDMA schedule is a real finite resource in the base paper's own model, and the delay it imposes is exactly computable. That's a defensible unit. It also bounds what we claim: no throughput in Mbps, no jitter, no multi-hop latency. *Evidence:* `Network._priority_phase` docstring; `CLAUDE.md` conventions.

Q. What's actually novel, and what isn't? *Short:* the findings are; the mechanisms mostly aren't. *Deeper:* not novel — trust in a metaheuristic fitness (P1, P2), Beta reputation (Jøsang 2002), credibility weighting (P2 has it), ECC pseudonyms (P3). Novel — the energy free ride, priority as a DoS surface, the coverage/latency opposition, crypto's computational dominance, and the reproduction audit. Plus a category the comparison papers have none of: published negative results. *Evidence:* `docs/related_work.md`.

Q. What would you remove if time runs out? *Short:* Module 4, the RSU-side IDS. *Deeper:* it's not started, nothing depends on it, and a 2026 paper (P2) already runs RF-style ML on the same public VeReMi dataset. Worse, VeReMi never simulated routing, so it contains none of the Layer-A attacks this project actually defends against — validating on it would prove little. *Evidence:* Part 30; `docs/related_work.md`.

Q. Aren't you worried the base paper's authors will scoop you? *Short:* on Module 1, yes — and it's handled by framing. *Deeper:* their §5 says they're already building the mobility-aware version of CSGD-NET. So Module 1 must be infrastructure, not the headline: "we first make CSGD-NET mobility-capable, as its authors indicate is needed." The security, priority and audit work is unaffected, and their paper would become motivation rather than competition. *Evidence:* base paper §5; `docs/related_work.md`.

Q. How do you know your later work didn't break your earlier results? *Short:* a regression gate over 55 configurations, checked at every step. *Deeper:* it hashes the raw float64 bytes of the energy vector, PDR counters, orphan and intra traces and trust scores. The invariant is that with `ems_frac=0` and zero crypto overhead, every pre-Module-5 result is bit-identical. It passed after every one of the six Module 5 steps, after Module 3a, and after the Module 3b config change. *Evidence:* `src/regression_gate.py`; `results/regression_reference.json`.

PART 38 — ONE-PAGE CHEAT SHEET

PROJECT CHIRP — Cuckoo-inspired Hierarchical Intrusion-Resilient Protocol for VANETs. HEAD `07c9aec`, 4 commits, clean, no remote.

PROBLEM Clustering saves energy by *fusing* readings at a cluster head. Fusion forces every reading to wait for the whole TDMA frame. Emergency traffic cannot wait — and the fast path that fixes it is a finite resource an attacker can steal.

BASE PAPER Sellami, Mchergui, Alaya, *Cluster Computing* 29:16 (2026), DOI 10.1007/s10586-025-05776-1. CSGD-NET: Cuckoo Search + Lévy + Gaussian walk, $f = E/(E_0 \cdot \text{dist}(\text{CH}, \text{RSU}))$.

CORE IDEA Make CSGD-NET mobile, trusted, authenticated and priority-aware — and measure what each costs.

MODULES 1 mobility+CH election ✓ · 2 trust ✓ · 3a Scyther ✓ · 3b crypto cost ✓ · 4 IDS ✗ **not started** · 5 priority ✓

ARCHITECTURE vehicle → cluster → CH → *fuse* → RSU. Emergency: vehicle → reserved slot → CH → *verbatim* → RSU.

KEY MODEL $E_{\text{TX}} = k \cdot E_{\text{elec}} + k \cdot \epsilon_{\text{fs}} \cdot d^2$ ($d \leq d_0$) or $k \cdot \epsilon_{\text{mp}} \cdot d^4$ ($d > d_0$); $d_0 = 87.71$ m. Delay: aggregated = cluster+1, bypassed = 2, orphan = 1 slots.

KEY PARAMS 100 nodes · E_0 0.5 J · 6400 bit · 22 frames/round · n_{ch} 10 · trust_gate 0.40 · deadline 4 slots · emergency_slots 1 · **sig_bits 576** · **mac_bits 128**

URBAN 600 m grid, 9 RSUs, tx 150 m · CHIRP orphan 3.8 %, HND +14.6 %, energy −11.7 % · bypass +1.8 % · attack drain 11.26 % @20 %

HIGHWAY 1 km 6-lane, 5 RSUs, tx 100 m · CHIRP FND +46 %, orphan −98.3 % · bypass +0.5 % · attack drain 6.01 % @20 % · detection 1.000

COVERAGE vs LATENCY deadline-miss tracks orphan rate **inversely**, 7 protocols × 2 scenarios. HEED (47 % orphan) best latency; CHIRP (0.1 %) worst.

ATTACK false-priority: assert on 22/22 not 4/22. 1 attacker saturates a cluster. @5 % attackers **83 %** of reservation energy serves liars. @20 % deadline-miss 0.437/0.472.

TRUST detection 0.94–1.00 — and with only a CH-eligibility gate it changed **nothing** (0.452→0.463). With a slot-grant gate it **halves** damage. @greed 0.50 detection **0.008**, below FPR, attacker still takes ~48 %.

AUTHORISATION **0.000 deadline-miss at every attacker fraction, both scenarios, $p = 5.1\text{e-}06$** . No false-positive cost.

SCYTHER v1.3.0, 32 verified / 8 by design. `priority.spdl` C4 Nisynch = Ok. **Found a real cross-protocol MAC-confusion bug** → domain-separation tags. `fusion-limit.spdl` fails by design → fusion loses provenance.

CRYPTO 64 B sig + 8 B digest, 122 B cert, 16 B MAC. sign 33.4 μs , **verify 97.4 μs (2.9×)**. Radio cost +1.63 % **urban** / +0.77 % **highway**; signature undetectable (p 0.94). **Computation $\approx 15\times$ radio** (extrapolated, $25\times/0.5$ W).

REPRODUCTION LEACH RMSE **0.005 ✓**. LEACH-C 0.049 / CSGD-NET 0.046 ✗ — **unreachable**: amplifier budget 0.002 J/node vs Table 4 spread 20–44×; zero-amplifier LEACH-C still 0.0108 short; drain rises +56 %/+60 %.

NOVELTY mechanisms mostly not novel. Contributions: energy free ride · priority as DoS · coverage↔latency · crypto is computational · reproduction audit. **Scoop risk: base authors are building mobility-aware CSGD-NET.**

LIMITATIONS synthetic mobility · no channel · slots not ms · Heinzelman at VANET range · computation energy not charged · unlinkability unverified · Module 4 absent.

STATUS Modules 1/2/3a/3b/5 complete, both scenarios, regression gate 55/55.

NEXT (1) regenerate the two stale urban JSONs; (2) cut or build Module 4; (3) 802.11p robustness check; (4) SUMO traces; (5) write-up. Fastest publication route: IEEE VNC short paper.

PART 39 — FINAL SANITY CHECK

39.1 Cross-checks performed

check	result
Every number cross-checked against <code>results/*.json</code>	done; two disagreements found and reported (Part 25.2, 25.3)
Config values read from <code>src/config.py</code> , not memory	done — full dump in Part 25.10
Git history read from <code>git log / git show</code> , not memory	done — Part 28
Module 4 state verified by search, not assumption	done — not started (Part 30)
Scyther counts verified by re-running the suite	done — 32/8
All doc-referenced file paths resolve	done — one false positive (<code>docs/module5.md</code> , referenced only to say it does not exist)
All runners execute (<code>--help</code> smoke test)	done — 5/5
Regression gate	55/55 identical

39.2 Uncertainties and explicit labels

- **ESTIMATED, not measured:** all OBU computation-energy figures. The $25\times$ slowdown and 0.25–2.0 W are assumptions, swept. Only the *timings on this machine* and the *sizes* are measured.
- **HYPOTHESIS, not fact:** that Table 4 was produced at a ~ 300 m field. The sweep supports it; the paper contradicts it; we do not claim it.
- **NOT INVESTIGATED and NOT ALLEGED:** how Table 4's values were produced. The linearity of HEED/PSO/GA and the non-monotonicity of LEACH-C/CSGD-NET are facts; the cause is not knowable from outside.
- **Bounded:** all Scyther results are "no attack within `--max-runs=4`".
- **Statistically undetected $\neq$ zero:** the per-message signature cost.
- **Directional only:** trust gating's false-positive cost at zero attackers.

39.3 Data-integrity defects found during this extraction

1. `results/module1_urban.json` — contains a 3-seed smoke test, not the 20-seed run reported in `docs/module1.md`. Docs authoritative.
2. `results/module2_urban.json` — contains a 3-seed smoke test, not the 12-seed run reported in `docs/module2.md`. Docs authoritative.
3. **Two result tables exist only in docs, with no committed runner:** the coverage/latency inversion (Part 8.6, 6 seeds) and the crypto decomposition (Part 22, 16 seeds). Both were produced by inline scripts.

Neither was fixed during this extraction, per the instruction not to modify the research while extracting it. All three are recorded as action items.

39.4 Things that exist only in conversation and are now on disk

Audited before this extraction and resolved in commit `07c9aec`: the three-paper comparison, the novelty verdict, the publication assessment, and the scoop risk all now live in `docs/related_work.md`; orientation lives in `CLAUDE.md`.

39.5 Still not on disk anywhere

- The **inline scripts** that produced the Part 8.6 and Part 22 tables. The numbers are documented; the code that produced them is not committed.

- The rejected hypotheses from the Scyther debugging are recorded as *outcomes* in Part 20 and the model headers, but the diagnostic scripts themselves (`t_verbatim2.spdl` , `t_agg.spdl` , `t_noTA.spdl`) were scratch and are not committed. `priority-assert.spdl` preserves the load-bearing one.
- The Scyther **binary** — deliberately, it is a build artifact.

39.6 Confidence statement

Every quantitative claim in this document traces to a named file in the repository or to a re-run of a named tool. Where a document and a data file disagree, both are shown. Where a figure is an extrapolation, it is labelled. Where a cause is unknown, it is left unknown.